

AMA 564 Deep Learning

2026 Spring

Lecture 10

- **Word Embedding**
- **Attention**
- **Transformer**

Word Embedding

- In natural language processing (NLP), a word embedding is a representation of a word.
- The embedding is used in text analysis.
- Typically, the representation is a real-valued vector that encodes the meaning of the word in such a way that words that are closer in the vector space are expected to be similar in meaning.

Extending to larger vocabularies

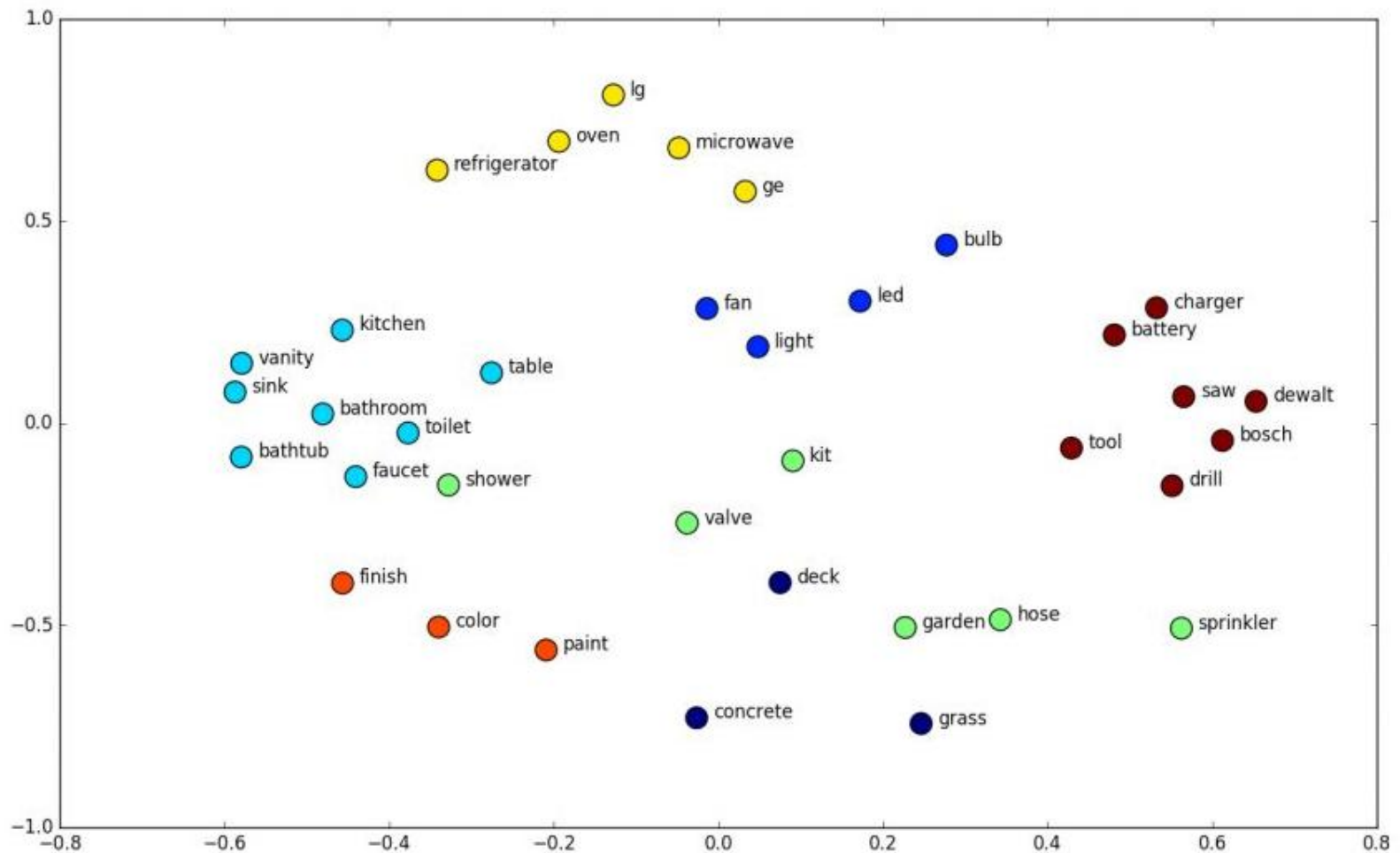
Word Embeddings Properties

- Word Similarities / Synonyms
- Linguistic Relationships

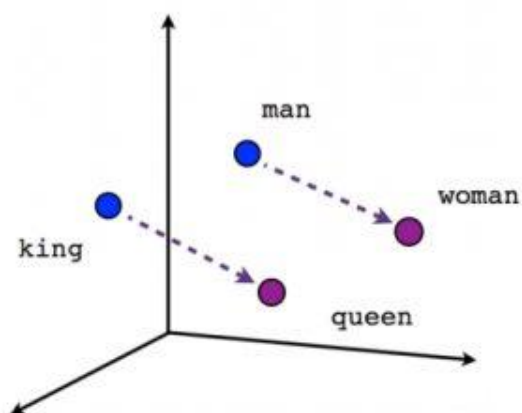
Word Similarities



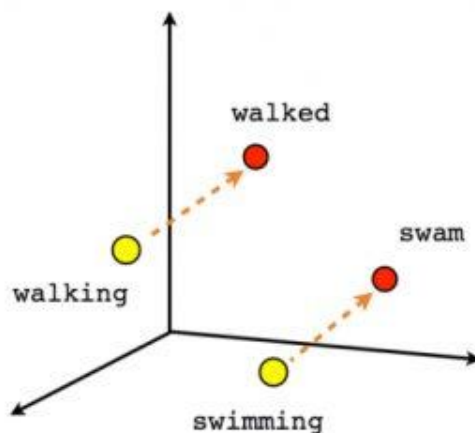
Example 2D word embedding space, where similar words are found in similar locations.



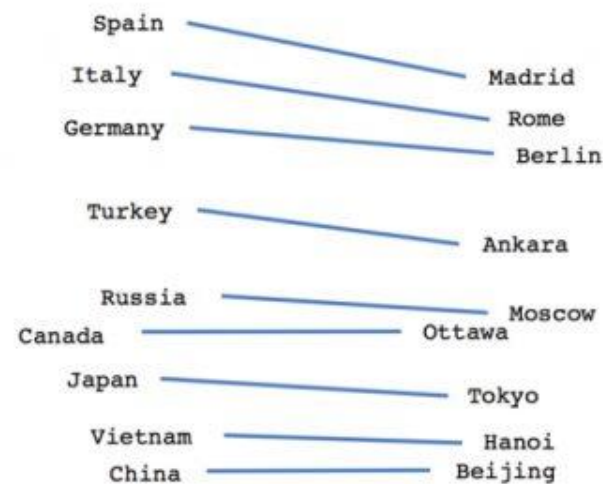
Source: <http://suriyadeepan.github.io>



Male-Female



Verb tense



Country-Capital

Three examples of relationships that are automatically uncovered during word-embedding training – male-female, verb tense, and country-capital.

Extending to larger vocabularies

The most popular algorithms include

1. Word2Vec algorithm from Google
2. GloVe algorithm from Stanford
3. fasttext algorithm from Facebook



- **Continuous Skip-gram Model:**
predicts words within a certain range before and after the current word in the same sentence.
- **Continuous Bag-of-Words Model (CBOW):**
predicts the middle word based on surrounding context words. The context consists of a few words before and after the current (middle) word. This architecture is called a bag-of-words model as the order of words in the context is not important.

To train a neural network with a single hidden layer to perform a prediction task. But the goal is to learn the weights of the hidden layer—these weights are the “word vectors”.



Skip-gram Model: Training data

“The wide road shimmered in the hot sun.”

Window Size	Text	Skip-grams
2	[The <u>wide</u> road shimmered] in the hot sun.	wide, the wide, road wide, shimmered
	The [wide road <u>shimmered</u> in the] hot sun.	shimmered, wide shimmered, road shimmered, in shimmered, the
	The wide road shimmered in [the hot <u>sun</u>].	sun, the sun, hot
3	[The <u>wide</u> road shimmered in] the hot sun.	wide, the wide, road wide, shimmered wide, in
	[The wide road <u>shimmered</u> in the hot] sun.	shimmered, the shimmered, wide shimmered, road shimmered, in shimmered, the shimmered, hot
	The wide road shimmered [in the hot <u>sun</u>].	sun, in sun, the sun, hot

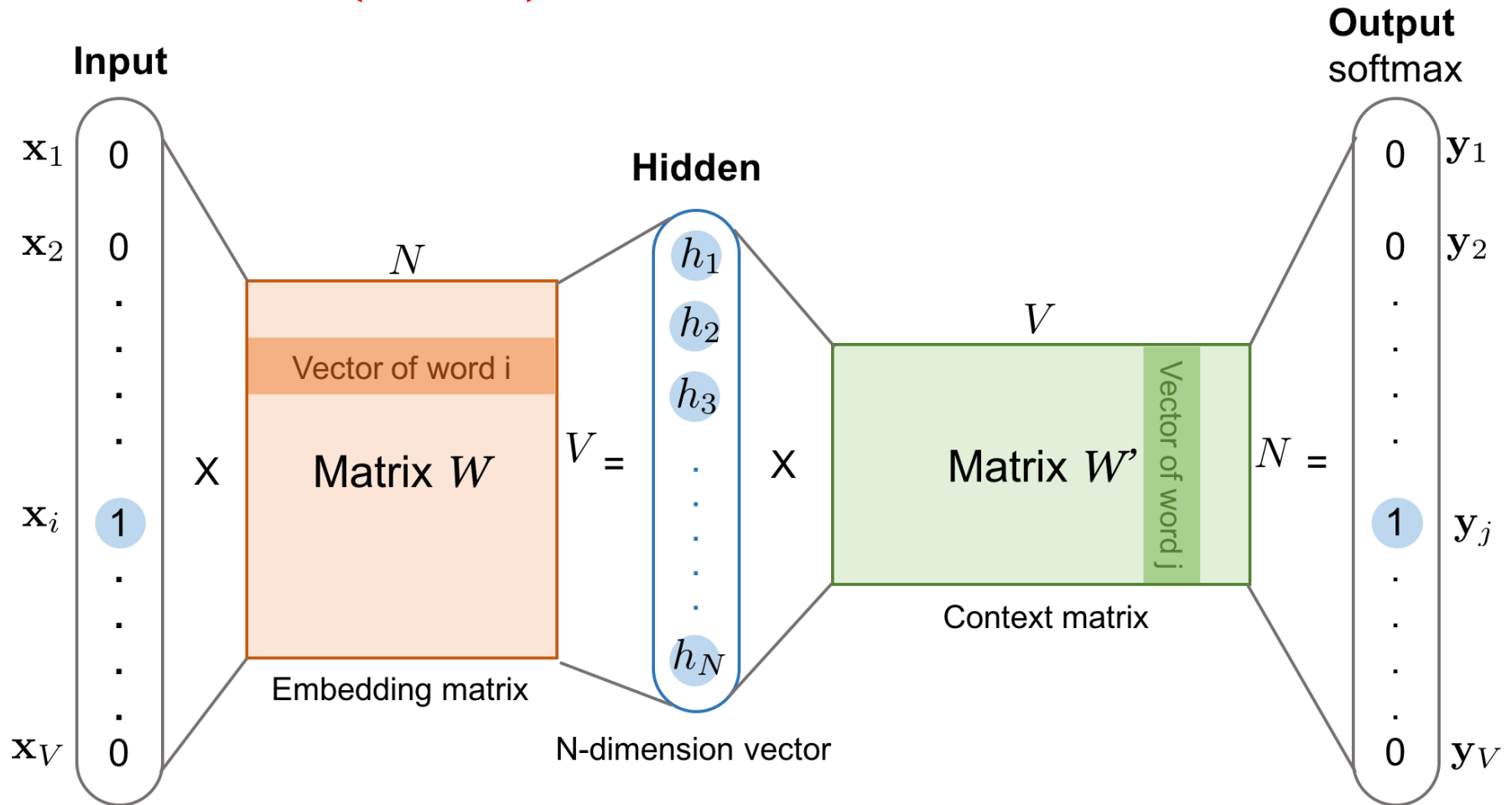


Skip-gram Model

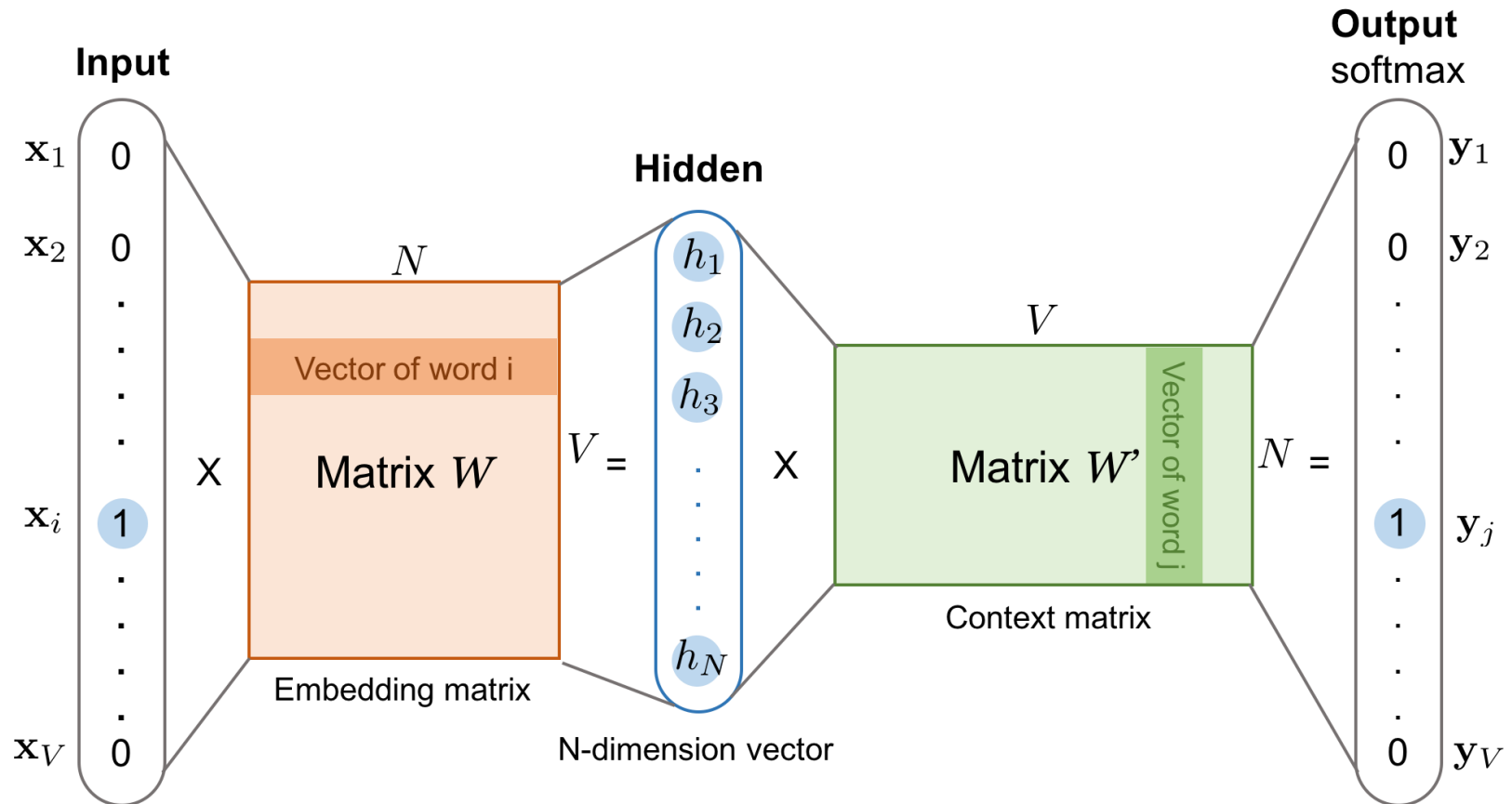
$x = (x_1, \dots, x_V)$ One-hot embedding of the input word

$y = (y_1, \dots, y_V)$ One-hot embedding of the target word

$\hat{y} = (\hat{y}_1, \dots, \hat{y}_V)$ Predicted vector of probabilities



Skip-gram Model



Rows of embedding matrix W are the embeddings of words.

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = \begin{bmatrix} 10 & 12 & 19 \end{bmatrix}$$

Rows of embedding matrix W are the embeddings of words.

“The quick brown fox jumps over the lazy dog.”

Window size: 2

Sentence

The quick brown fox jumps over the lazy dog. →

The quick brown fox jumps over the lazy dog. →

The quick brown fox jumps over the lazy dog. →

The quick brown fox jumps over the lazy dog. →

Training Samples

CBOW

((quick, brown), the)

((the, brown, fox), quick)

((the, quick, fox, jumps), brown)

((quick, brown, jumps, over), fox)

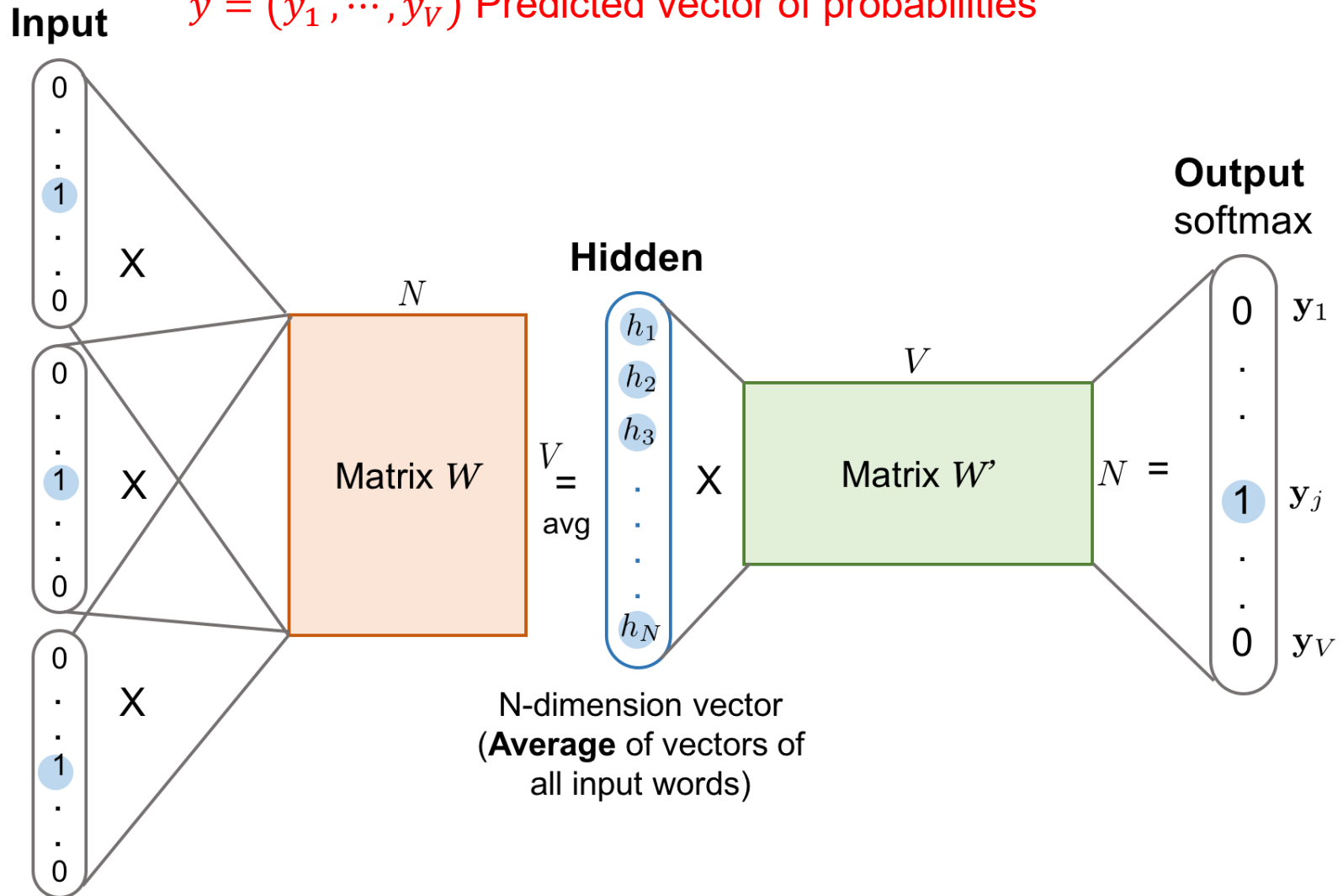
CBOW Model

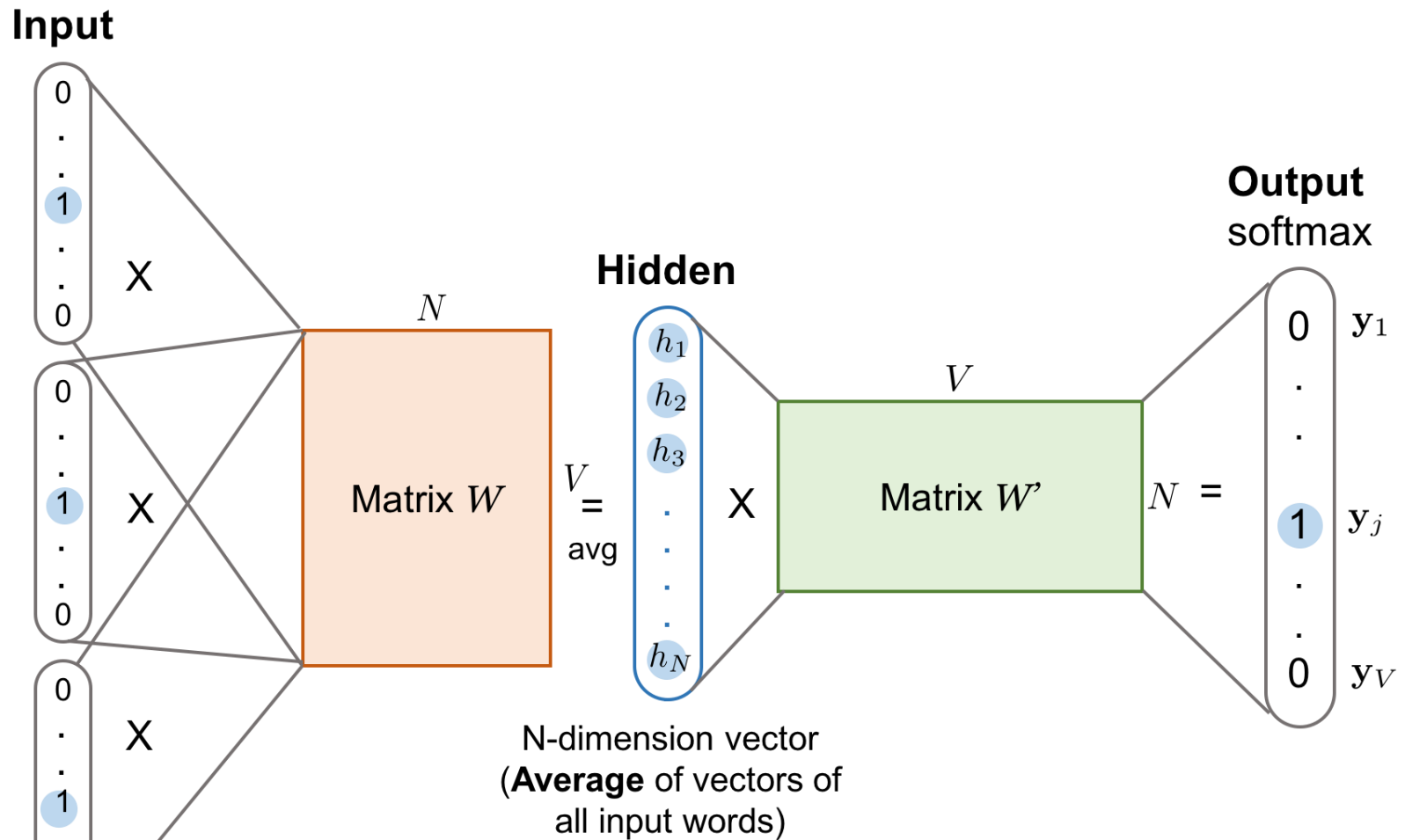


$X_1, X_2, \dots, X_w$ A bag of One-hot embeddings of the input words

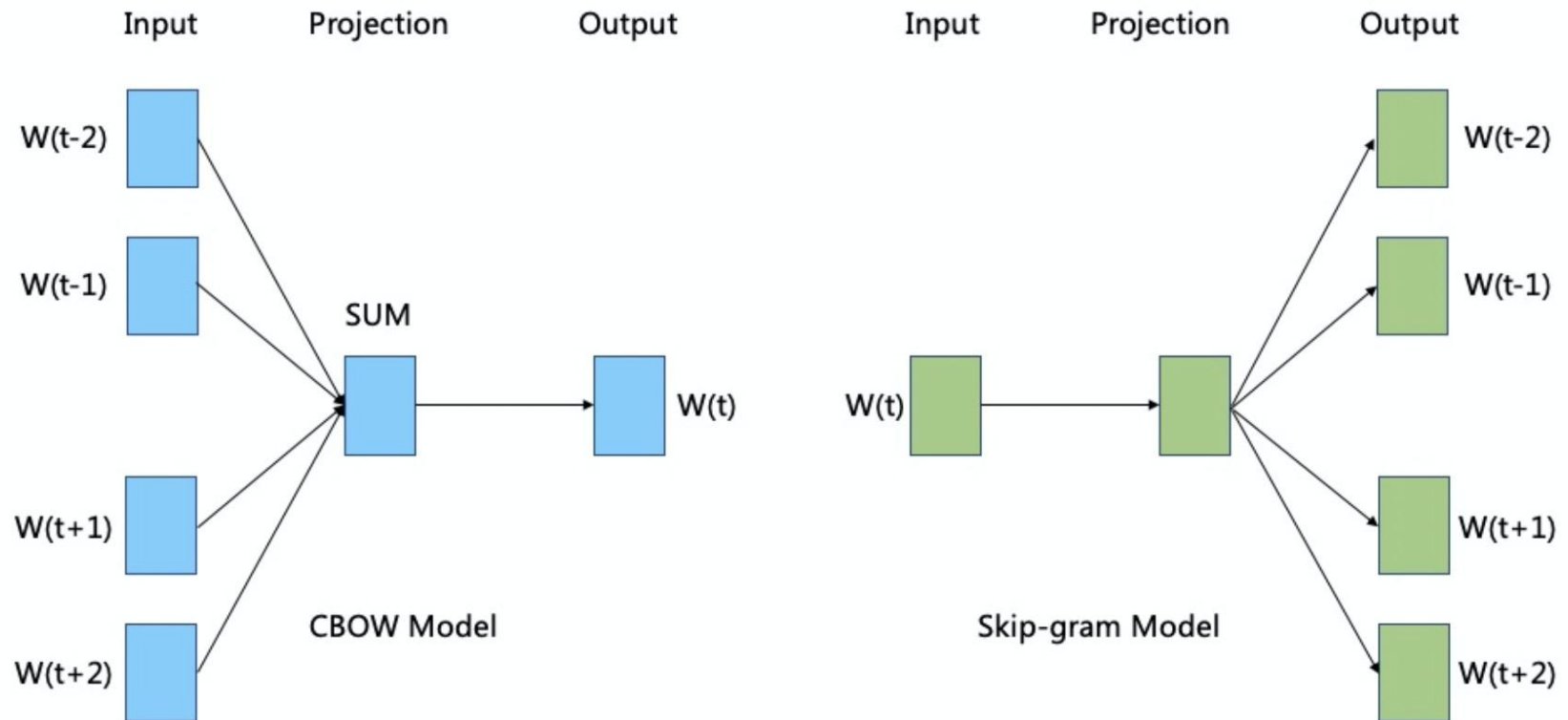
$y = (y_1, \dots, y_V)$ One-hot embedding of the target word

$\hat{y} = (\hat{y}_1, \dots, \hat{y}_V)$ Predicted vector of probabilities





Rows of embedding matrix W are the embeddings of words.



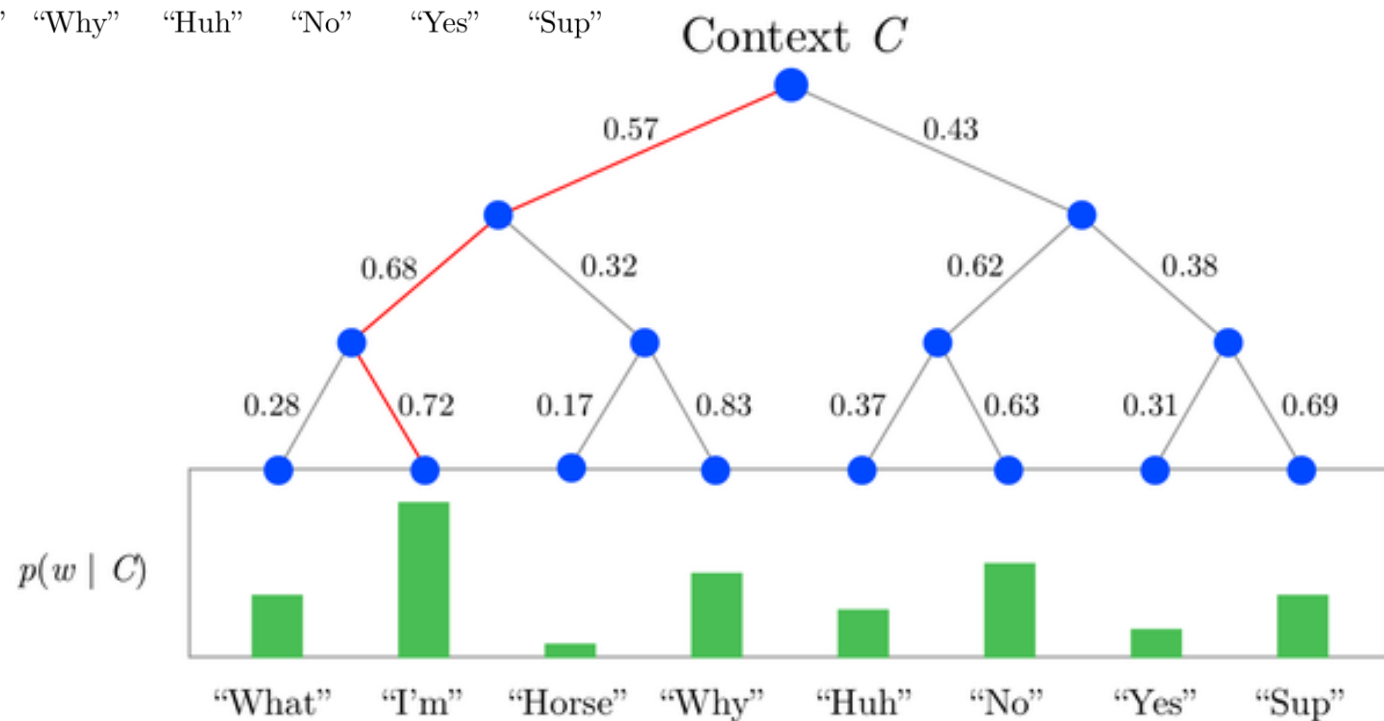
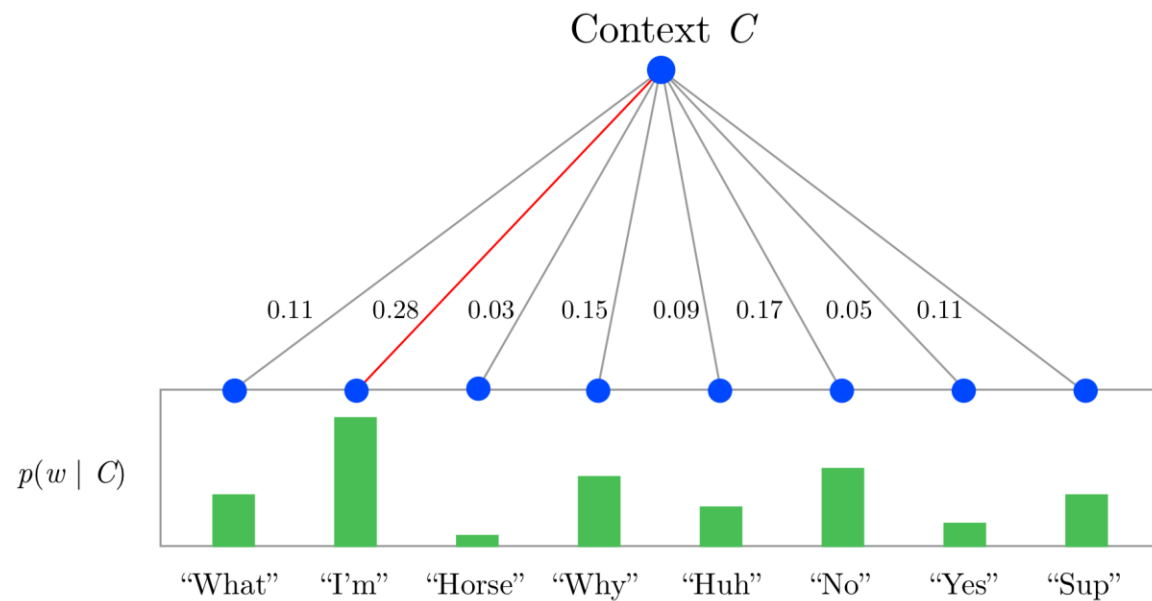
Word2Vec, GloVe, FastText

<https://www.youtube.com/watch?v=9S0-OC4LFNo>

When vocabulary is extremely large, calculating the denominator of SoftMax is computationally impractical.

- **Hierarchical Softmax** (Morin and Bengio ([2005](#)))
- **Negative Sampling** (Mikolov et al. ([2013](#)))

Hierarchical Softmax



Computed V times
for all vocabs

$$\left\{ \begin{bmatrix} p(w_1|w^{(t)}) \\ p(w_2|w^{(t)}) \\ p(w_3|w^{(t)}) \\ \vdots \\ p(w_V|w^{(t)}) \end{bmatrix} \right\} = \frac{\exp(W_{output} \cdot h)}{\sum_{i=1}^V \exp(W_{output(i)} \cdot h)} \in \mathbb{R}^V$$

Complexity = $O(V + V) \approx O(V)$
where V is very large

V computations are needed to
get normalization factor

$$\begin{bmatrix} p(D=1|w, c_{pos}) \\ p(D=1|w, c_{neg,1}) \\ p(D=1|w, c_{neg,2}) \\ p(D=1|w, c_{neg,3}) \\ \vdots \\ p(D=1|w, c_{neg,K}) \end{bmatrix} = \frac{1}{1 + \exp(-(\{c_{pos}\} \cup W_{neg}) \cdot h)} \in \mathbb{R}^{K+1} \quad (7)$$

Negative Sampling



Vanilla Skip-Gram

$$W_{\text{output (old)}} - \boxed{0.05} \times \text{grad_W_output} = W_{\text{output (new)}}$$

-0.560	0.340	0.160
-0.910	-0.440	1.560
-1.210	-0.130	-1.320
1.670	-0.150	-1.030
1.720	-1.460	0.730
0.000	1.390	-0.120
-0.060	1.520	-0.790
0.800	1.850	-1.670
-1.370	1.320	-0.480
0.670	1.990	-1.850
-1.520	-1.740	-1.860

(11X3)

0.064	0.071	-0.014
0.098	0.015	0.063
0.069	0.089	0.045
0.014	0.085	0.079
-0.021	0.067	0.071
-0.098	-0.088	0.091
-0.072	-0.078	-0.089
0.046	-0.079	-0.053
-0.049	-0.087	0.025
-0.060	0.092	0.042
0.074	0.050	0.070

(11X3)

-0.563	0.336	0.161
-0.915	-0.441	1.557
-1.213	-0.134	-1.322
1.669	-0.154	-1.034
1.721	-1.463	0.726
0.005	1.394	-0.125
-0.056	1.524	-0.786
0.798	1.854	-1.667
-1.368	1.324	-0.481
0.673	1.985	-1.852
-1.524	-1.743	-1.864

(11X3)

Negative Sampling

$$W_{\text{output (old)}} - \boxed{0.05} \times \text{grad_W_output} = W_{\text{output (new)}}$$

-0.560	0.340	0.160
-0.910	-0.440	1.560
-1.210	-0.130	-1.320
1.670	-0.150	-1.030
1.720	-1.460	0.730
0.000	1.390	-0.120
-0.060	1.520	-0.790
0.800	1.850	-1.670
-1.370	1.320	-0.480
0.670	1.990	-1.850
-1.520	-1.740	-1.860

(11X3)

Learning R.

0.05

×

Not computed!		
---------------	--	--

(11X3)

Positive sample, w_o
 Negative sample, k=1
 Negative sample, k=2
 Negative sample, k=3

0.031	0.030	0.041
-0.090	0.031	-0.065
0.056	0.098	-0.061
0.069	0.084	-0.044

(11X3)

-0.560	0.340	0.160
-0.910	-0.440	1.560
-1.210	-0.130	-1.320
1.670	-0.150	-1.030
1.720	-1.460	0.730
0.000	1.390	-0.120
-0.060	1.520	-0.790
0.798	1.849	-1.672
-1.366	1.318	-0.477
0.667	1.985	-1.847
-1.523	-1.744	-1.858

(11X3)

Binomial Classification

(regression, logistic)

(regression, machine)

(regression, sigmoid)

(regression, supervised)

(regression, neural)

VS

aegis4048.github.io

(regression, zebra)

(regression, pimples)

(regression, Gangnam-Style)

(regression, toothpaste)

(regression, idiot)

Likely to observe

$$p(D = 1|w, c_{pos}) \approx 1$$

Unlikely to observe

$$p(D = 1|w, c_{neg}) \approx 0$$

Negative Sampling



Drilling fluids serve to balance formation pressures while drilling to ensure wellbore stability. They also carry cuttings to the surface and cool the bit. The drilling **engineer** traditionally designs drilling fluids with two primary goals in mind:

- To ensure safe, stable boreholes, which is accomplished by operating within an acceptable mud-weight window
- To achieve high rates of penetration so that rig time and well cost can be **minimized**

current context word

These **primary** considerations do not include well productivity **concerns**. A growing recognition of the importance of drilling-induced formation damage has **led** to the use of drill-in fluids (fluids used to drill through the pay zone) that minimize formation damage. This **page** discusses the formation damage that may be associated with various types of drilling fluids.

Center word: **drilling**

$K = 5$

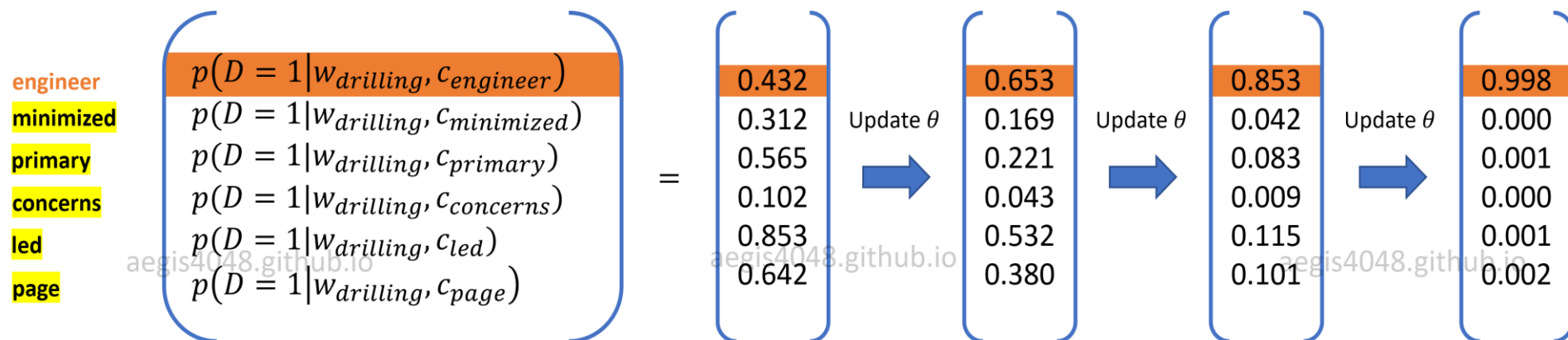
$iter = 1$

$iter = 2$

$iter = 3$

$iter = 4$

Current context word: **engineer**



Attention

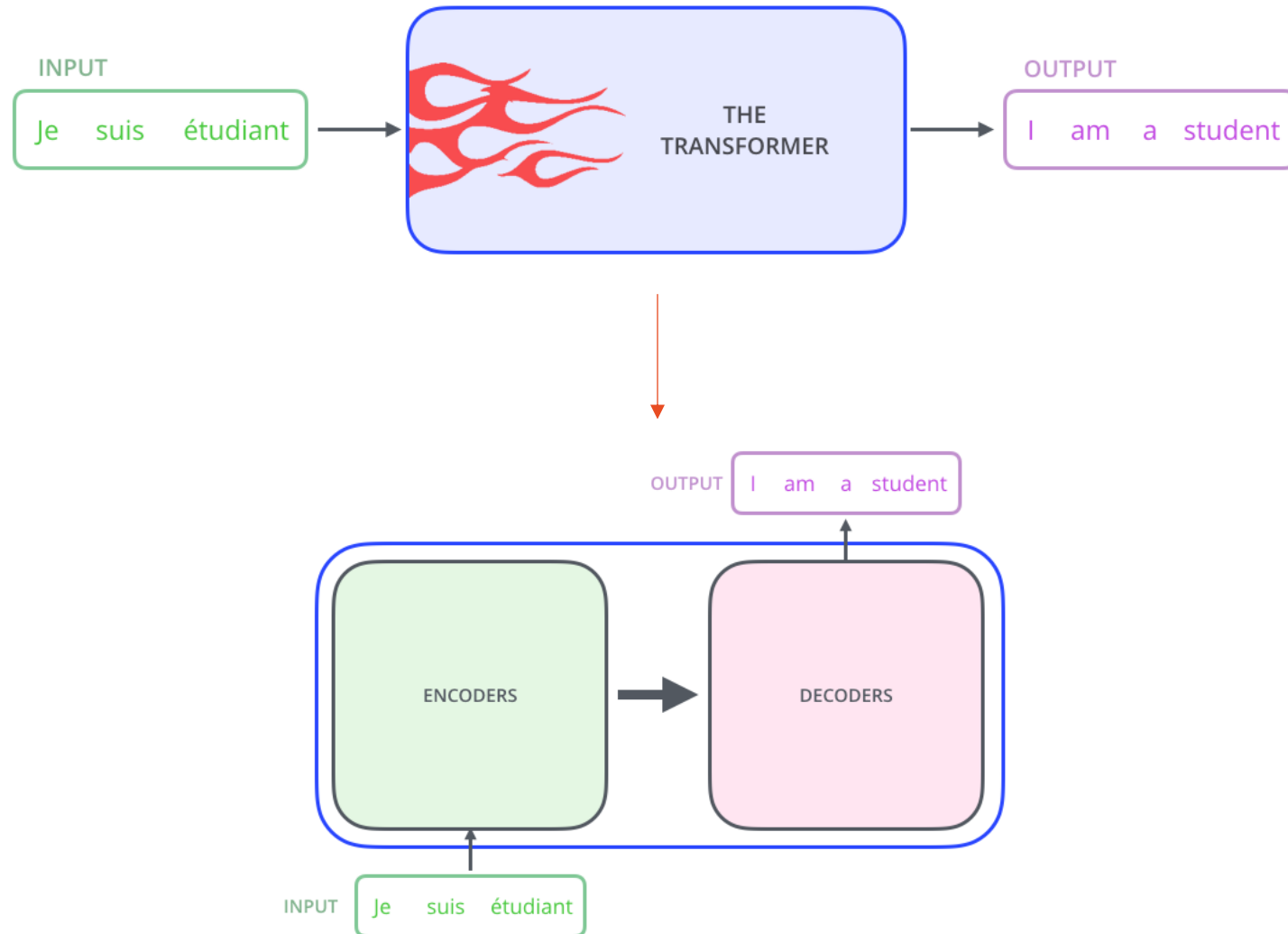
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

Google citation: **172734**

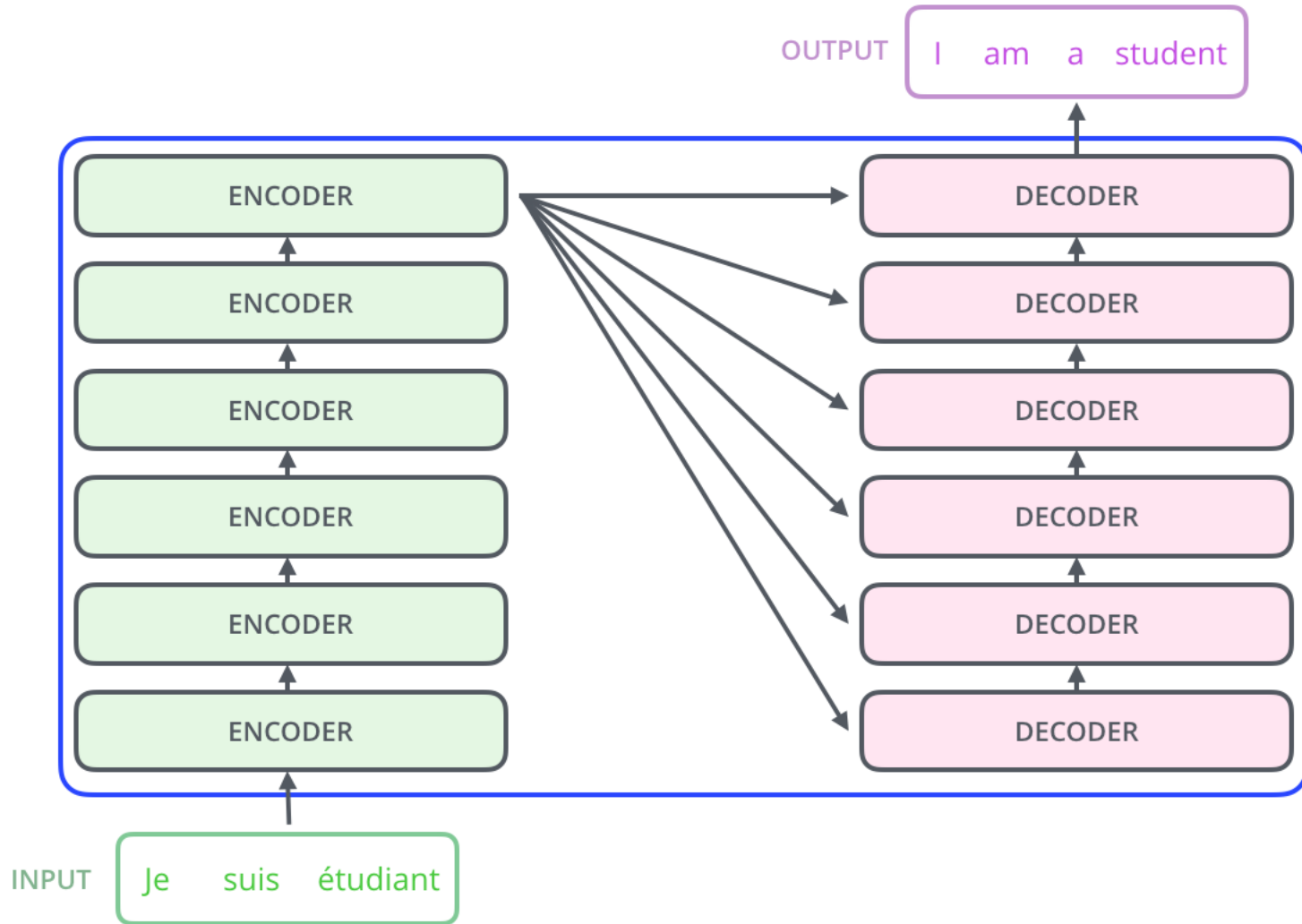
A high-level look at the translation task



A high-level look at the translation task



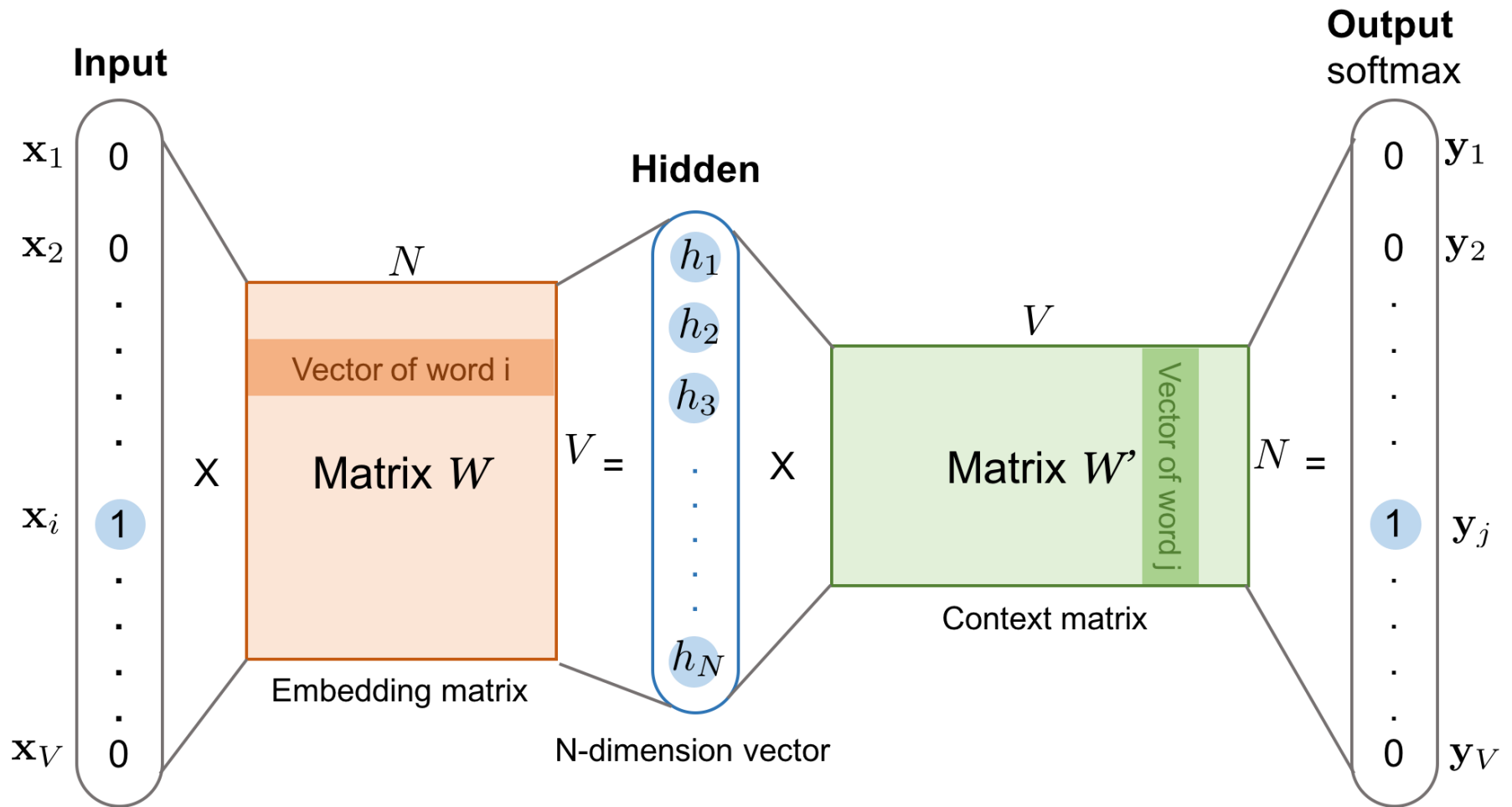
A closer look



Encode the input information (obtain the embedding)

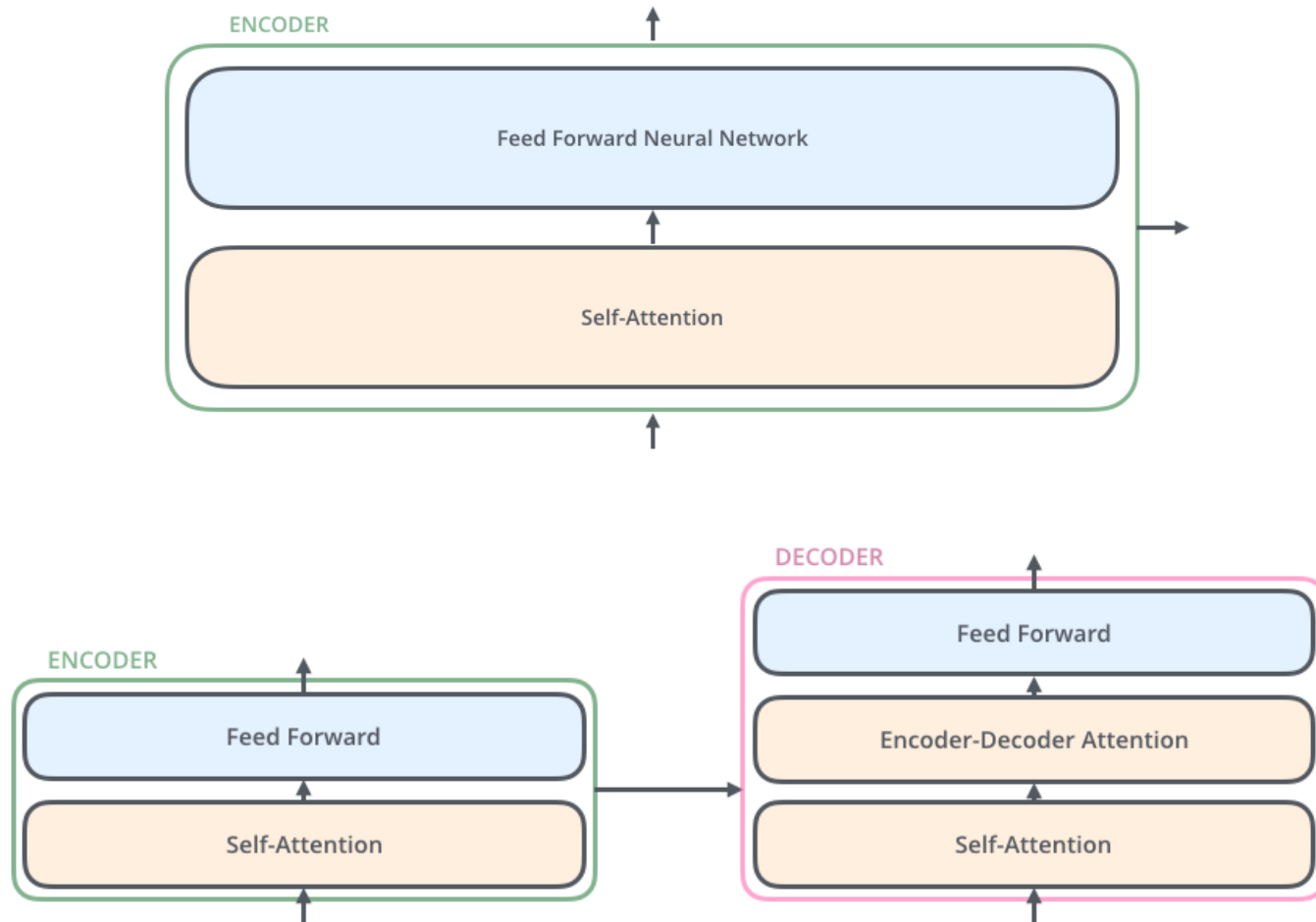
Decode the information (predict words)

The structure of encoder and decoder ?



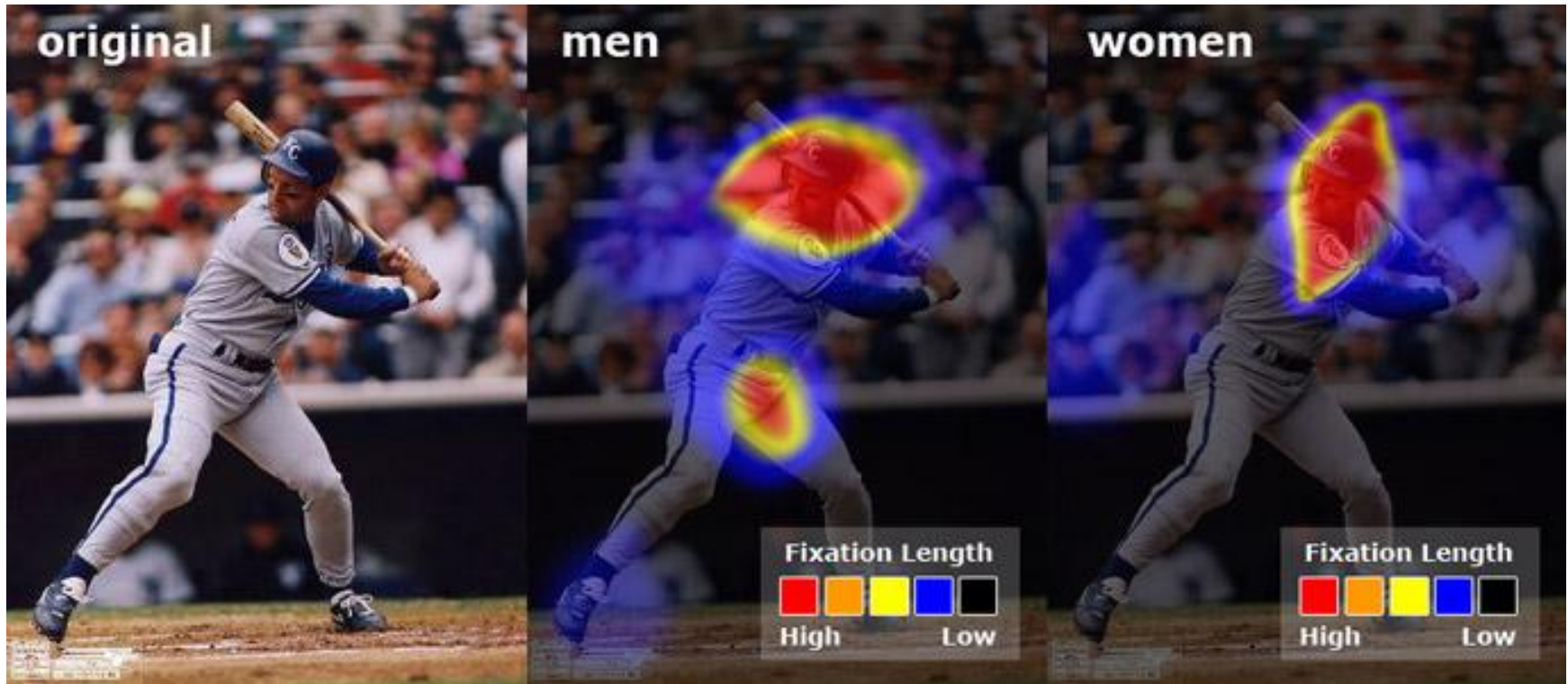
What are the disadvantages of MLP?

Encoder/Decoder with attention in high-level



The additional attention layer is to help the model focuses on the right parts of the data!

Attention example



source: <https://packageinsight.com/the-importance-of-visual-attention/>

Attention example

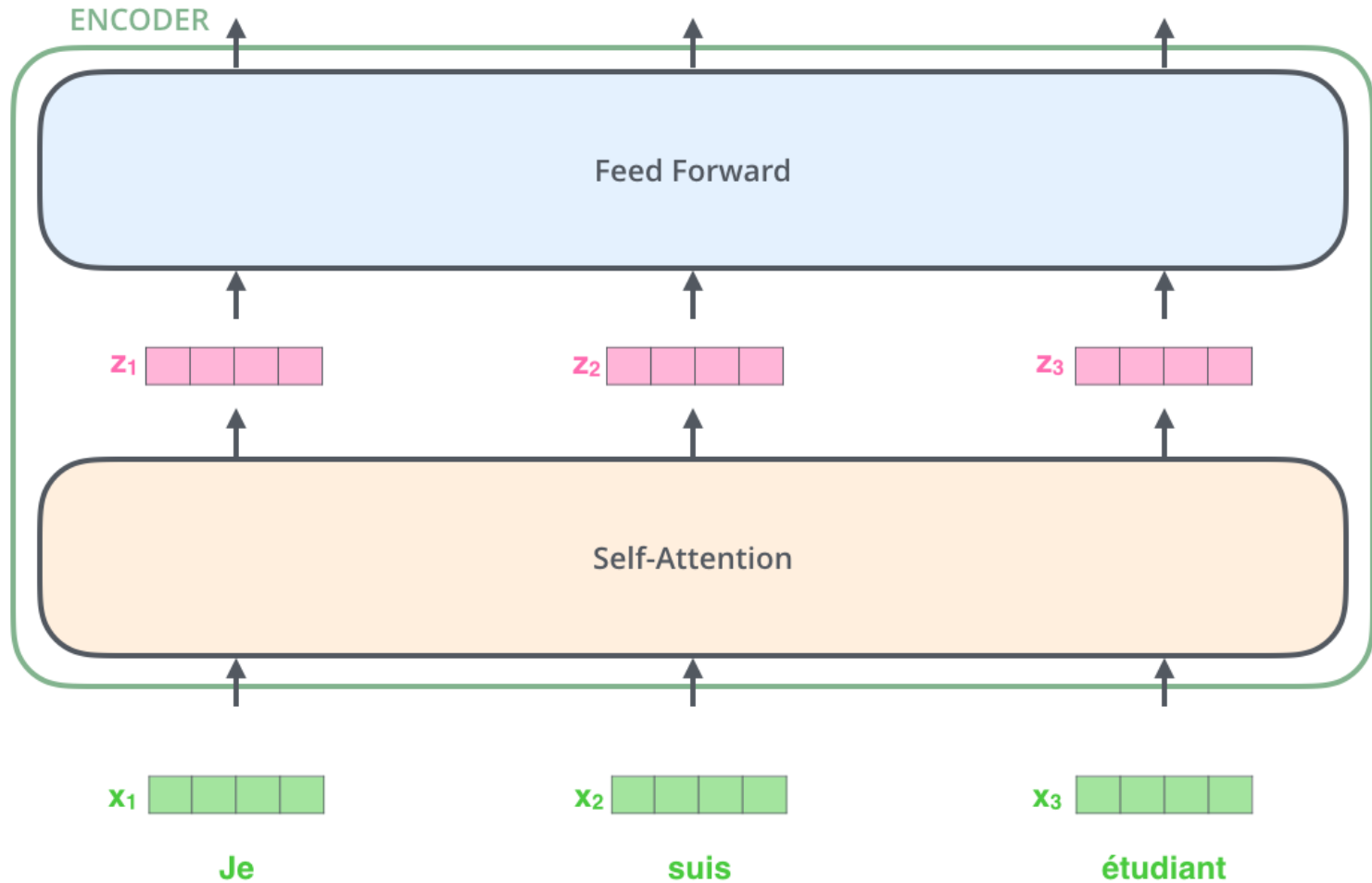


source: <https://zhuanlan.zhihu.com/p/91839581>

Attention example

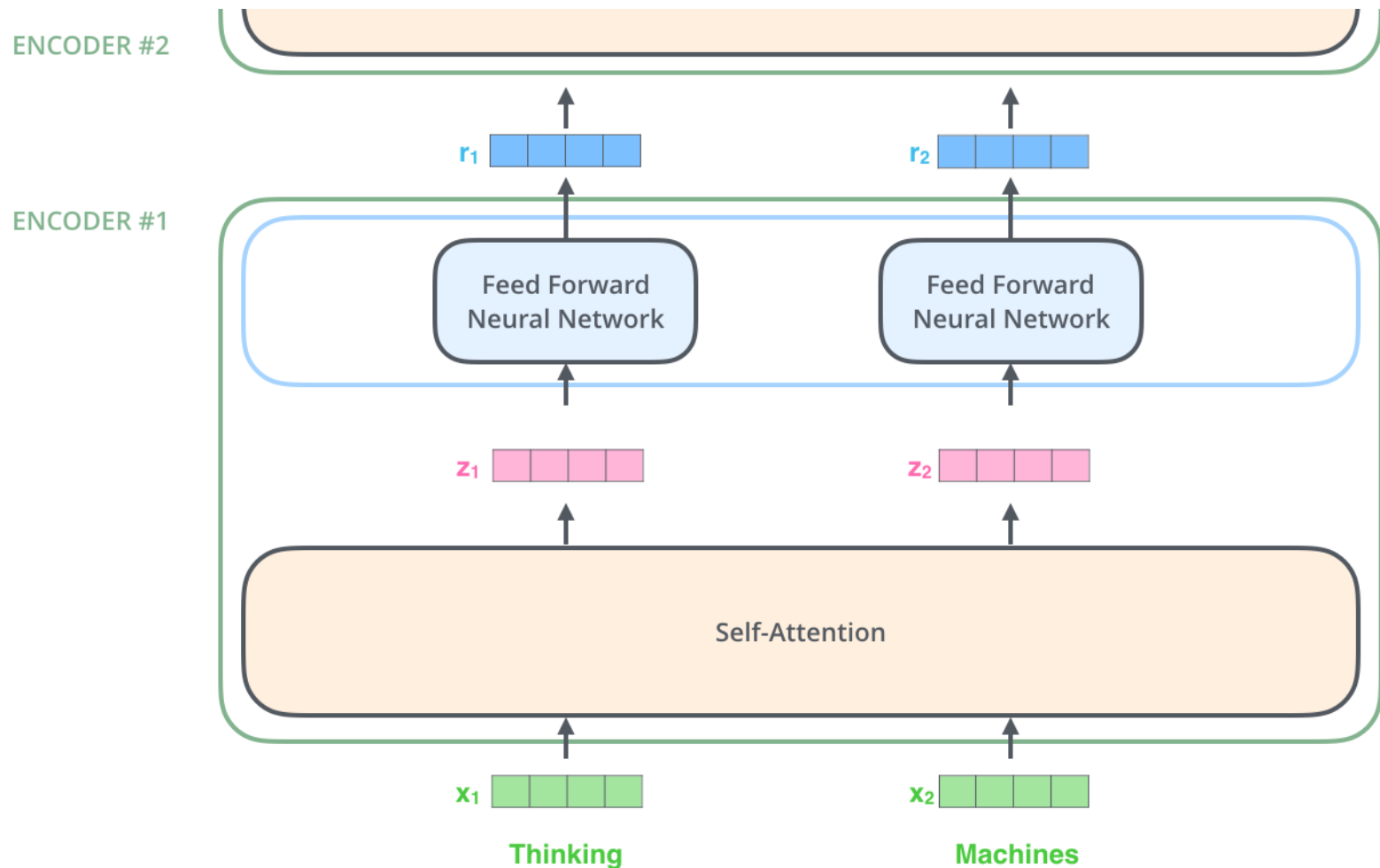


A little bit more details



Conceptually, the attention layer will give some weights to the embedding of each word.

From one-layer to another



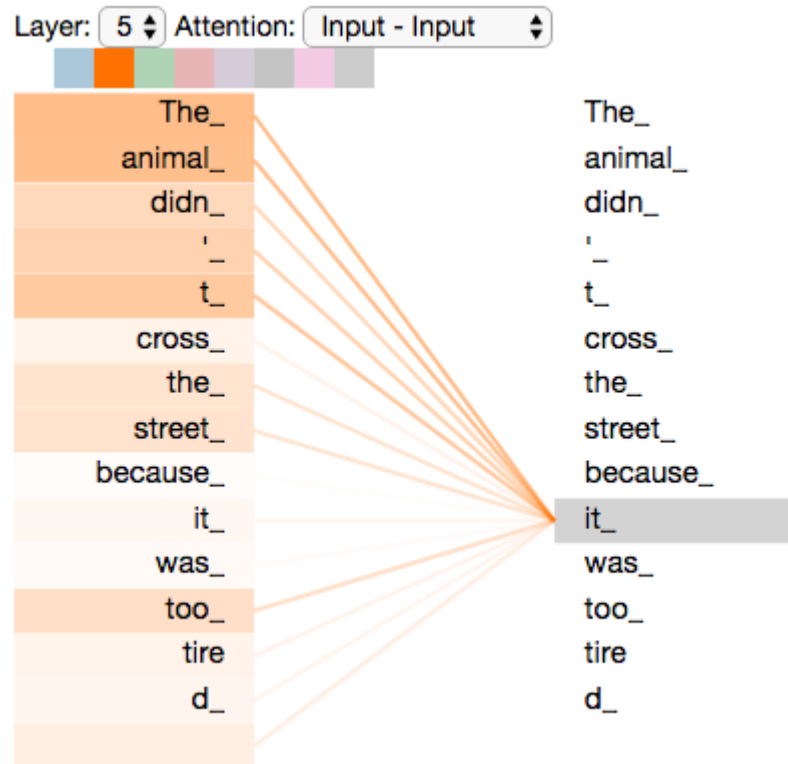
The word at each position passes through a self-attention process. Then, they each pass through a feed-forward neural network -- the exact same network with each vector flowing through it separately.

Consider translating the following sentence:

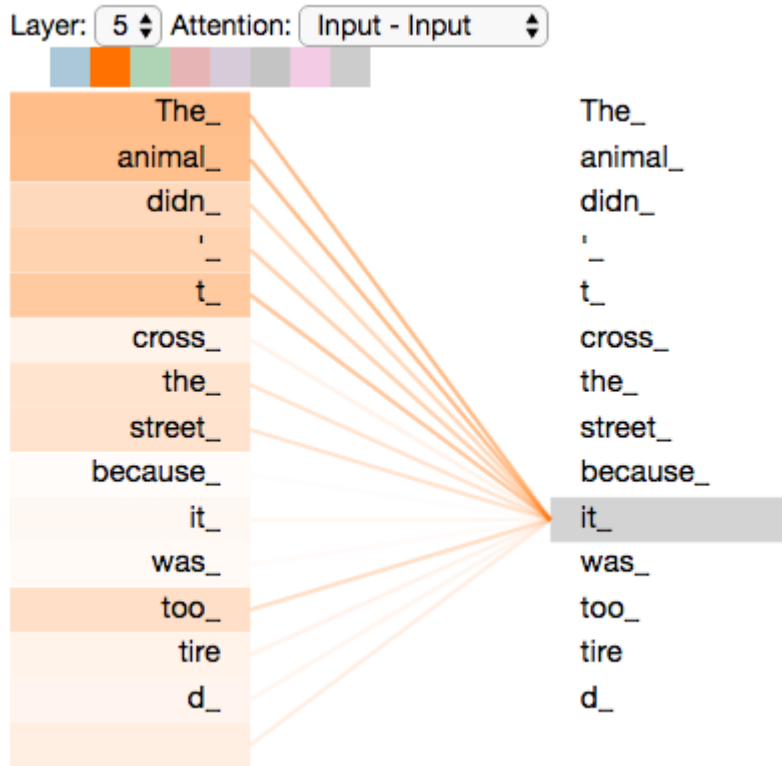
“The animal didn't cross the street because it was too tired”

Question: What does the word “it” refer to?

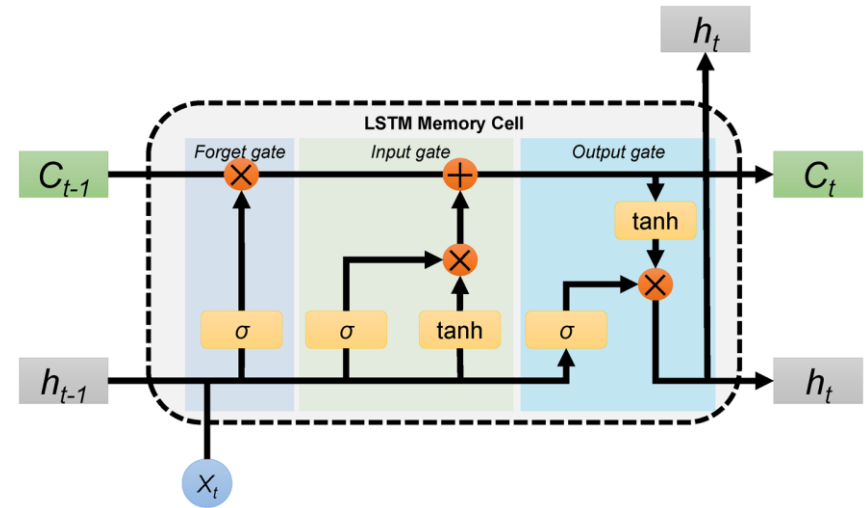
The context is very important!!!



Compare self-attention to LSTM

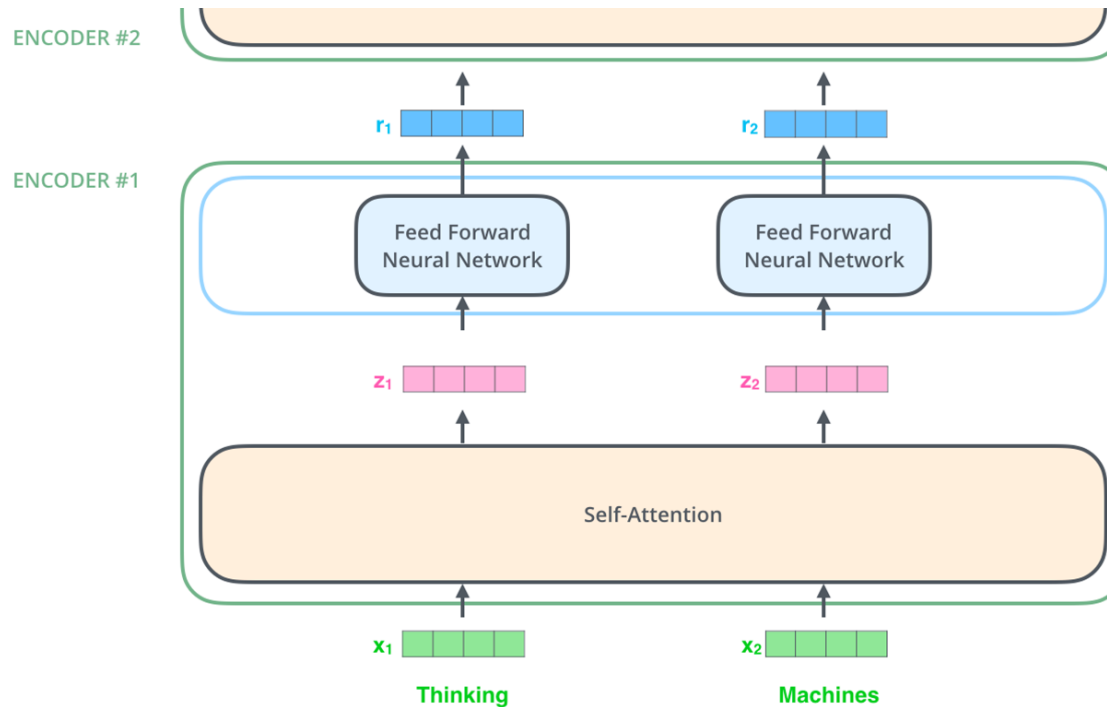


Self-attention



LSTM

A detailed illustration of self-attention



Step 1: Given the embedding x , we will compute the **Query** vector, the **Key** vector, and the **Value** vector.

How to create Q, K, V?

These vectors are created by multiplying the embedding by three matrices **that we trained** during the training process.

Self-attention in details: An overview of Q, K, V

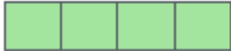


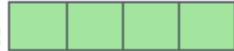
Input

Thinking

Machines

Embedding

x_1 

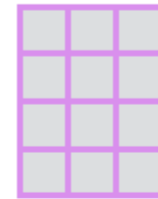
x_2 

(Dimension 64)

Queries

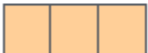
q_1 

q_2 



W^Q

Keys

k_1 

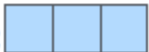
k_2 



W^K

Values

v_1 

v_2 

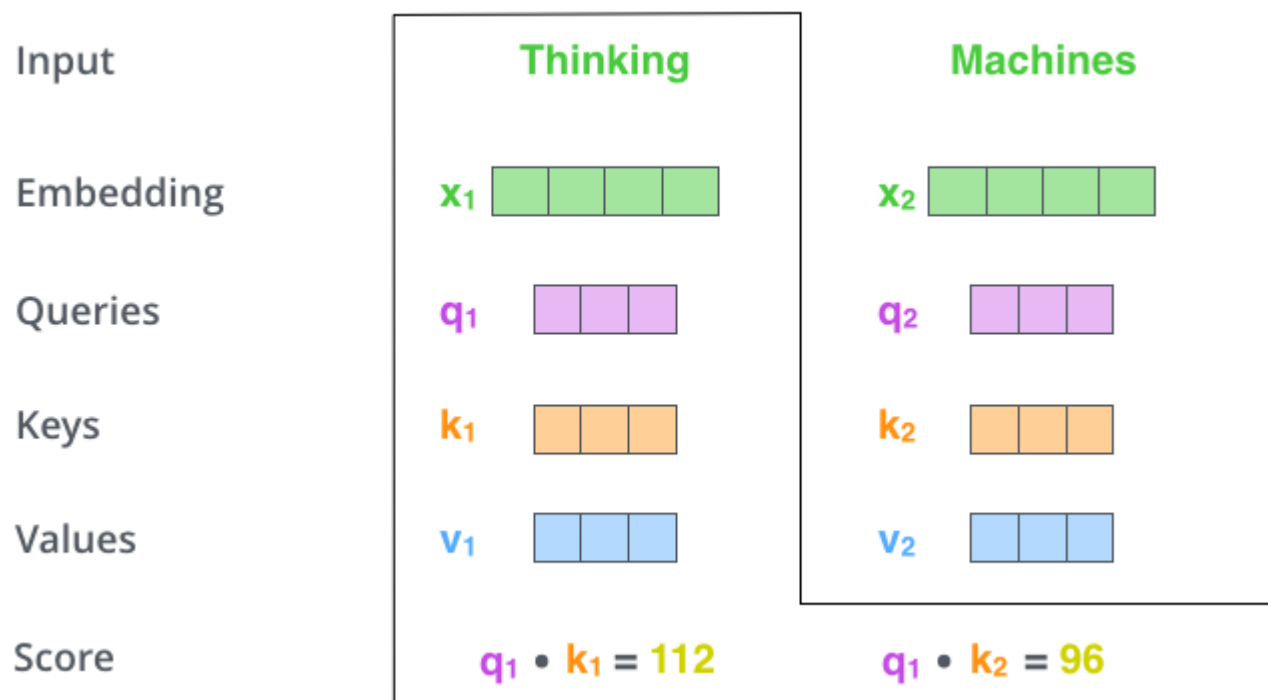


W^V

Step 2: Compute similarities of the Q and Keys



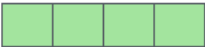
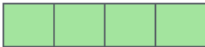
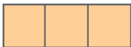
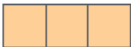
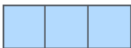
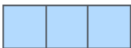
Step 2: The score is calculated by taking the dot product of the **query vector** with the **key vector** of the respective word we're scoring.



Step 3: Normalization



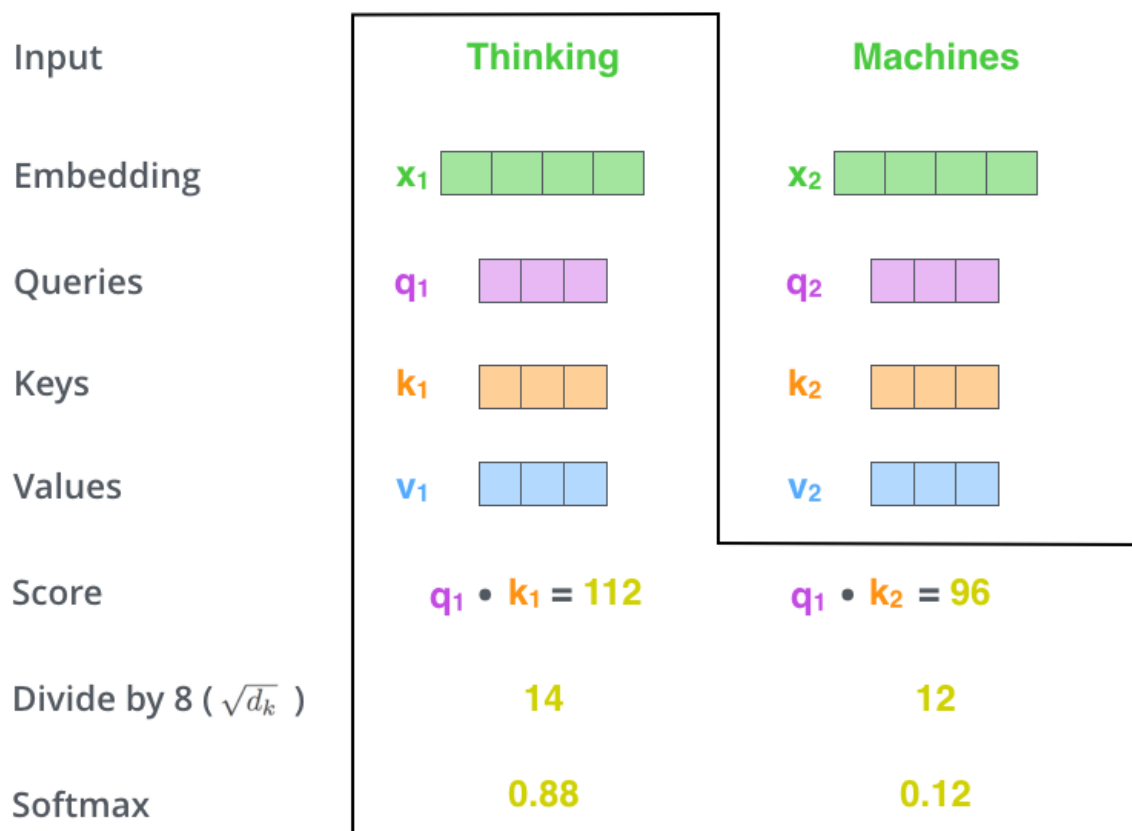
Step 3: normalize the score by $\sqrt{d_k}$, where $\sqrt{d_k}$ is the dimension of the **key vector**.

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Step 4: Normalization by Softmax



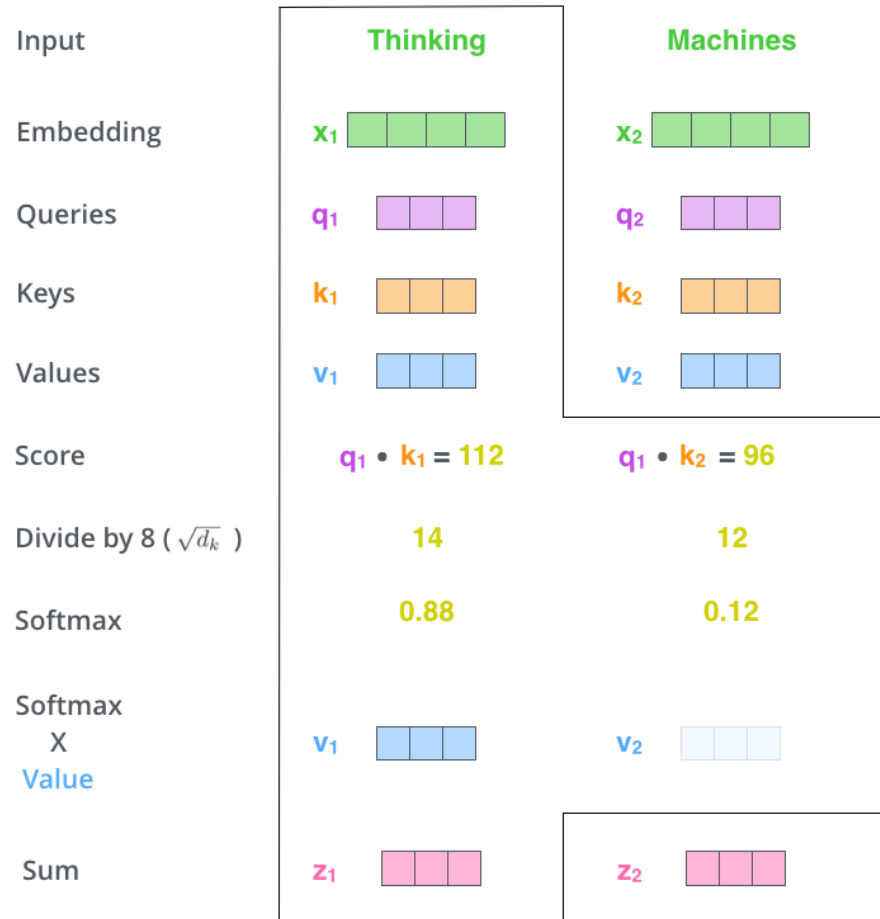
Step 4: Further normalize the score by softmax.



Step 5: Reweight the value vector by the attention score



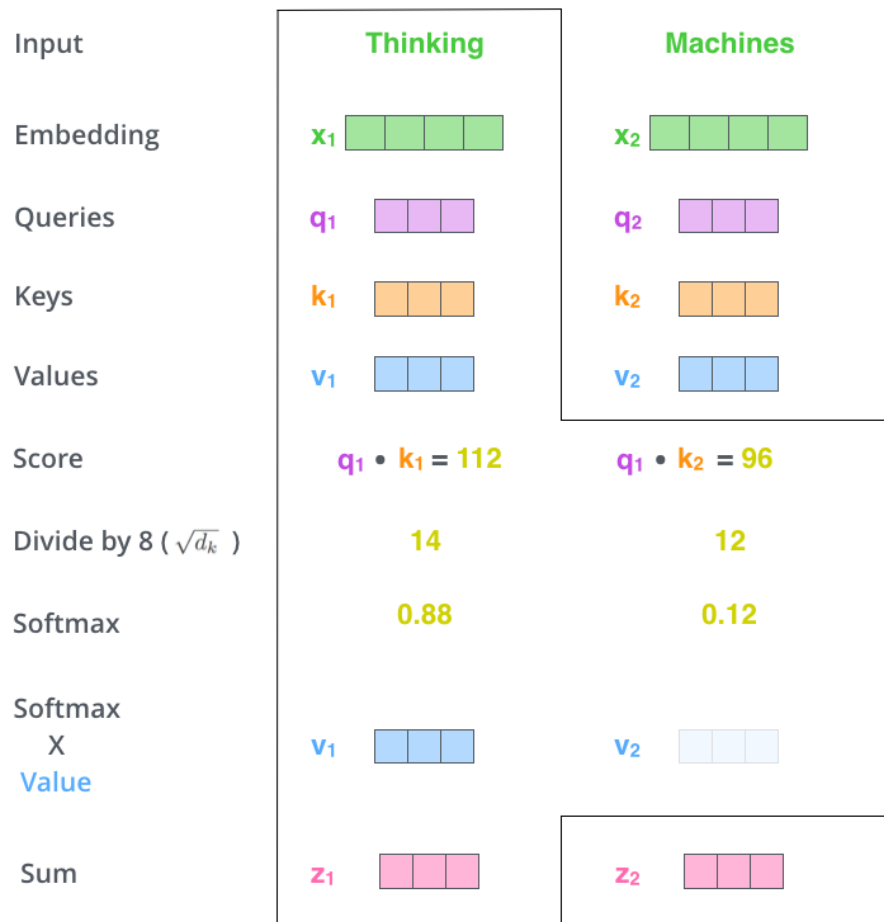
Step 5: Multiply each value vector by the softmax score (in preparation to sum them up).



Step 6: Create the output of the attention layer



Step 6: Sum up the weighted value vectors. This produces the output of the self-attention layer at this position (for the first word).



Put all of them together in a matrix form



Create the Q, K, V: Every row in the X matrix corresponds to a word in the input sentence. Here, W^Q , W^K , W^V are trained parameters.

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^Q \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^K \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{K} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^V \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

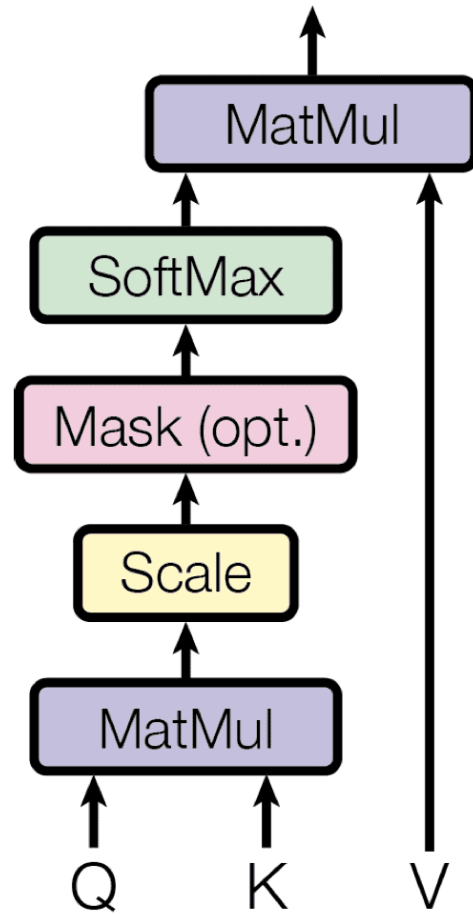
$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline & \\ \hline & \\ \hline & \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

=

Z

$\begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array}$

The Architecture in the paper



Attention Animation

https://www.bilibili.com/video/BV1dt4y1J7ov/?spm_id_from=333.337.search-card.all.click&vd_source=cc88065420e6f003010306f4a620f7f3

Attention in Neural Networks

<https://www.youtube.com/watch?v=W2rWgXJBZhU>

Attention Mechanism In a nutshell

<https://www.youtube.com/watch?v=oMeIDqRguLY>

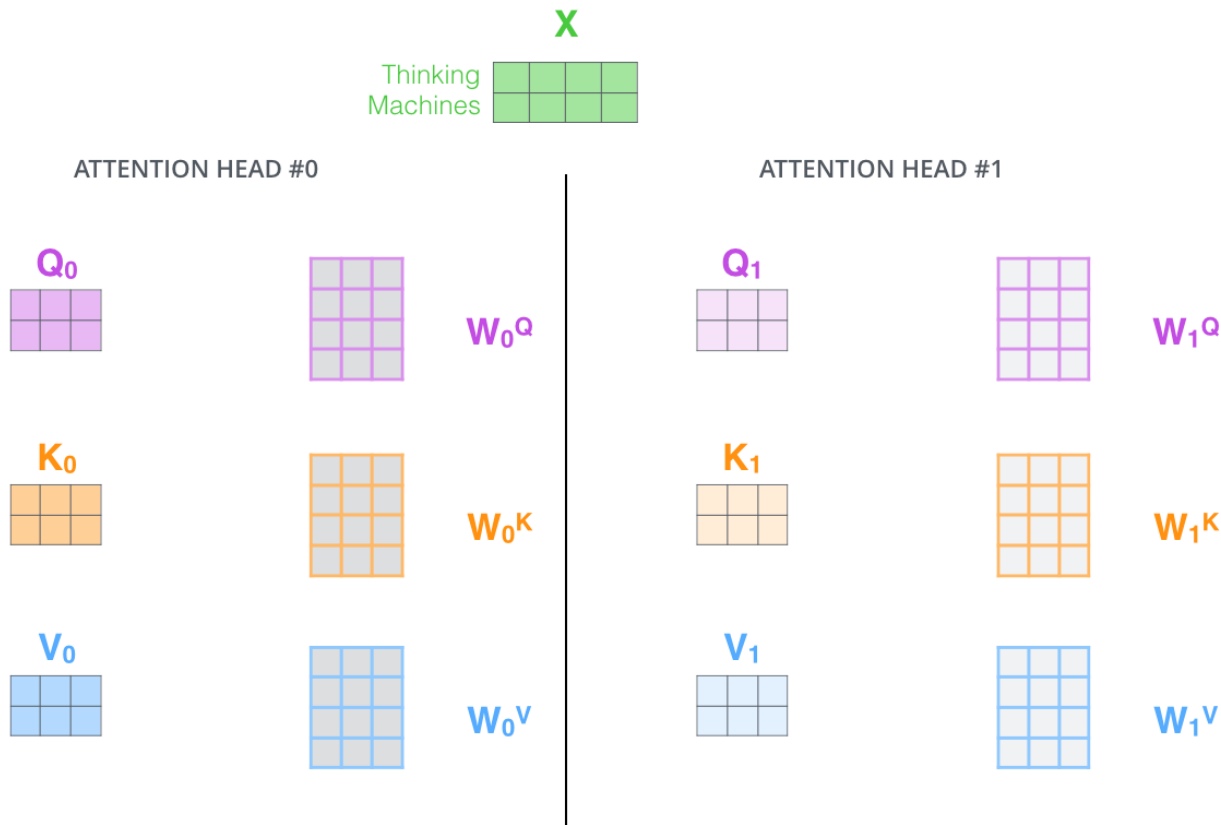
The math behind Attention: Keys, Queries, and Values matrices

https://www.youtube.com/watch?v=UPtG_38Oq8o



Transformer

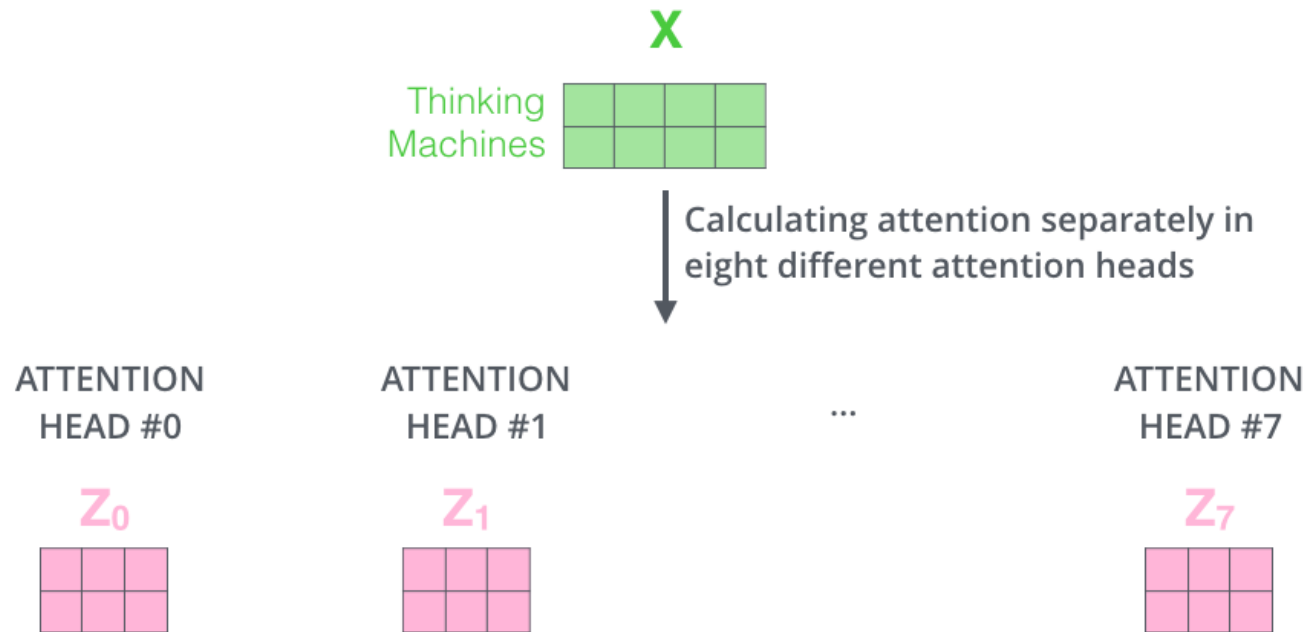
From one attention to multi-heads attention



Motivations:

1. Increase capability for focusing on different positions of the input sequence data.
2. Increase the representation power of the value vectors.

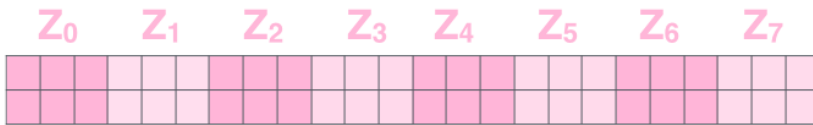
Multi-heads attention in action



How to aggregate information of the multi-heads?



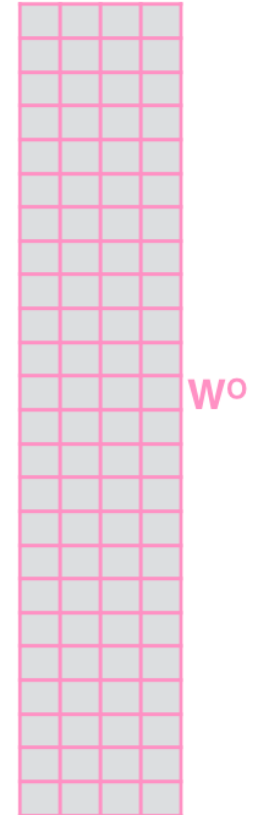
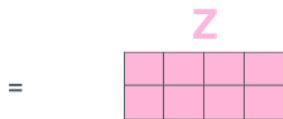
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$

3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



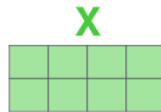
Multi-heads attention in one picture



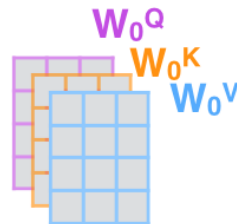
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



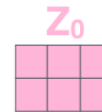
3) Split into 8 heads.
We multiply X or R with weight matrices



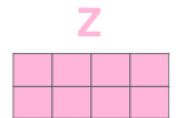
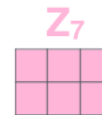
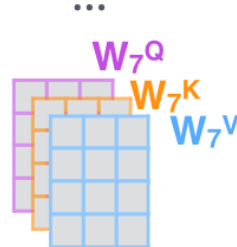
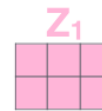
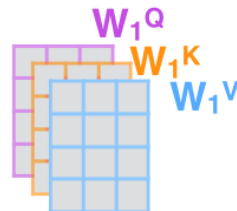
4) Calculate attention using the resulting $Q/K/V$ matrices



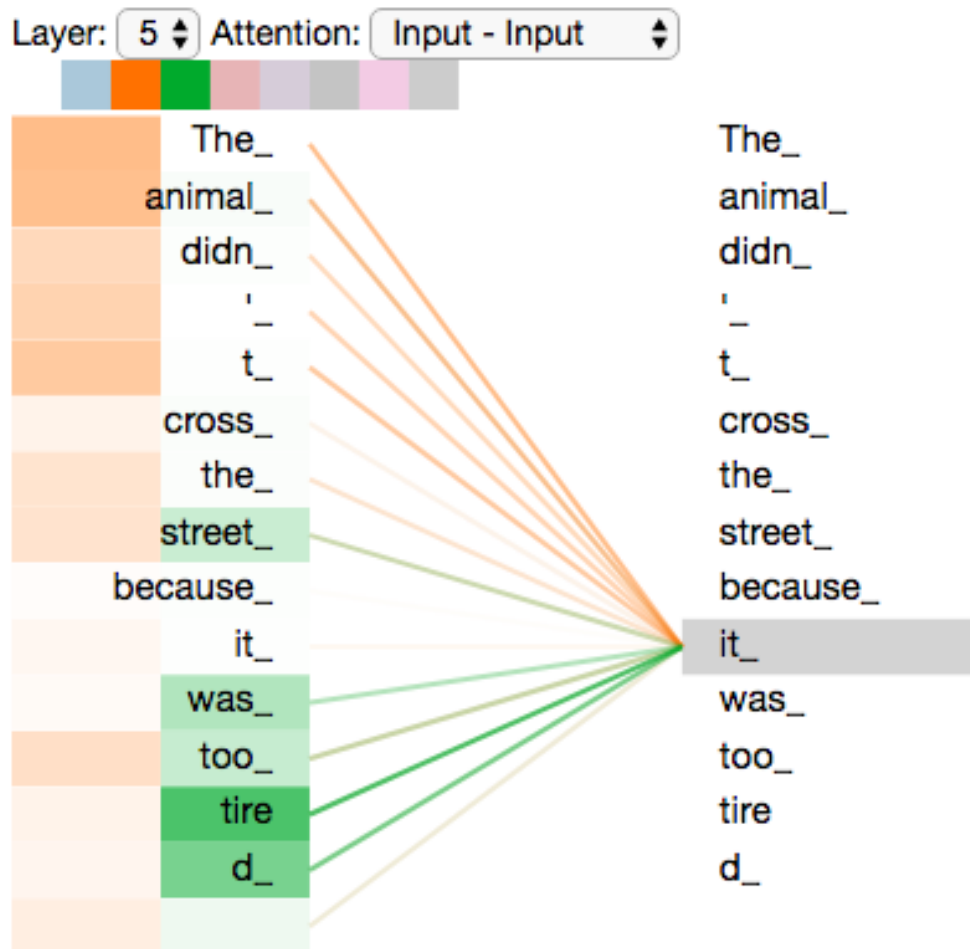
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



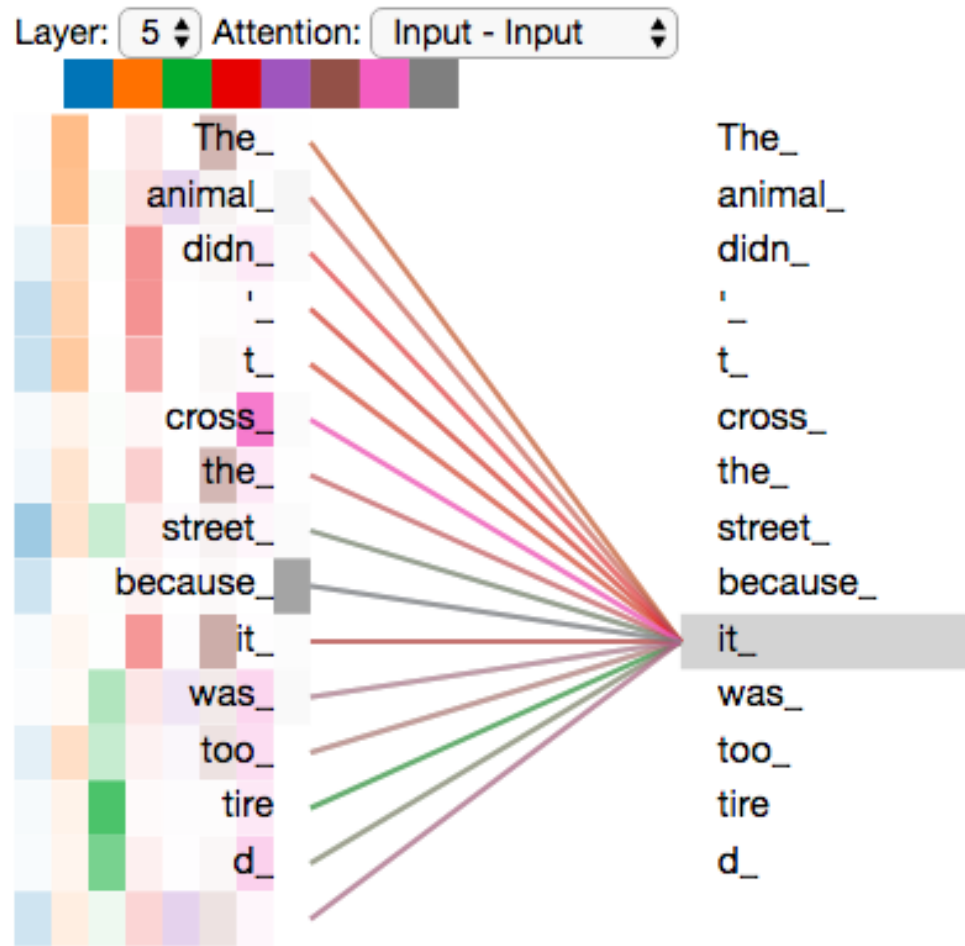
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



Multi-heads attention in practice

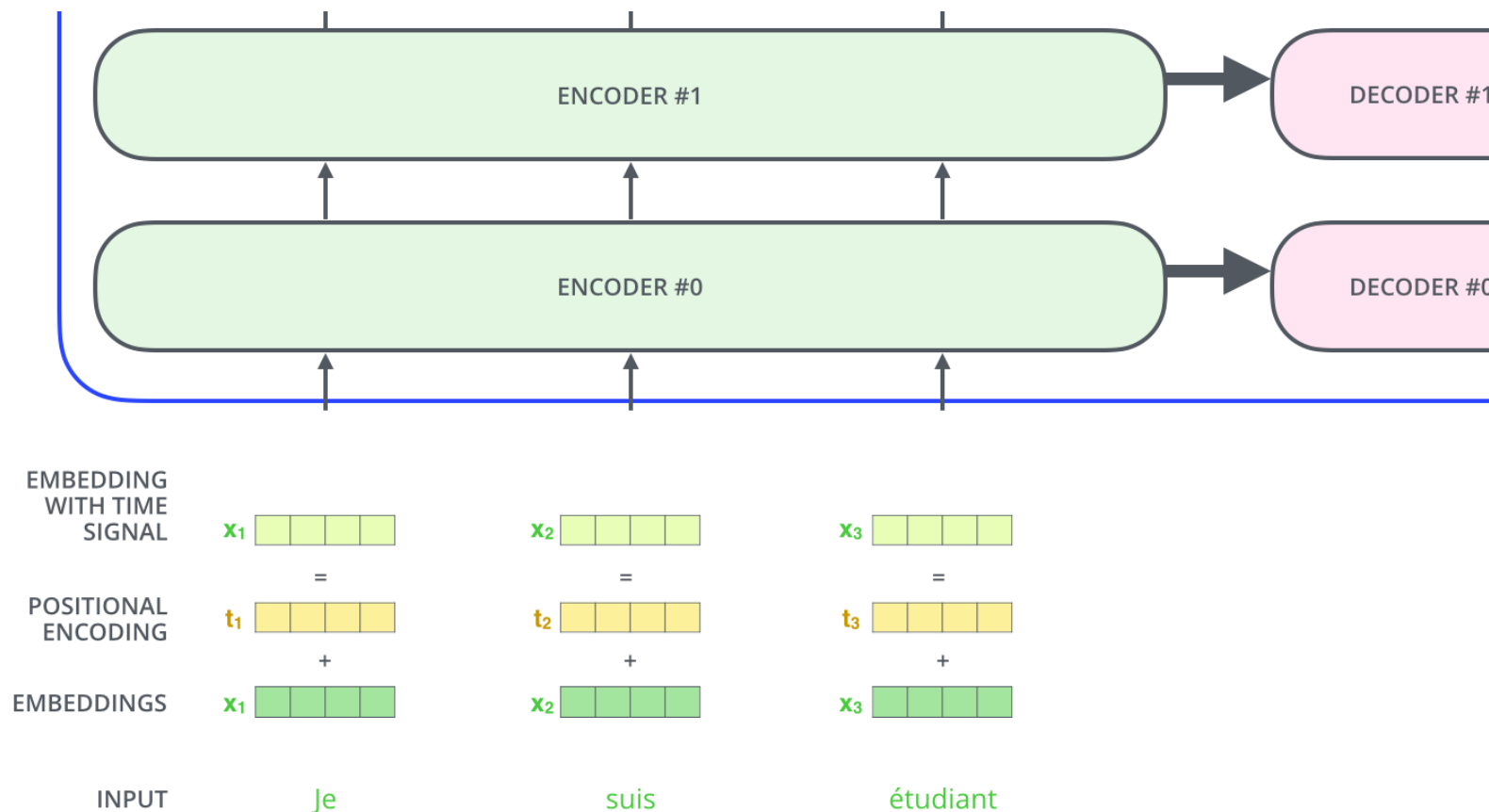


Multi-heads attention in practice



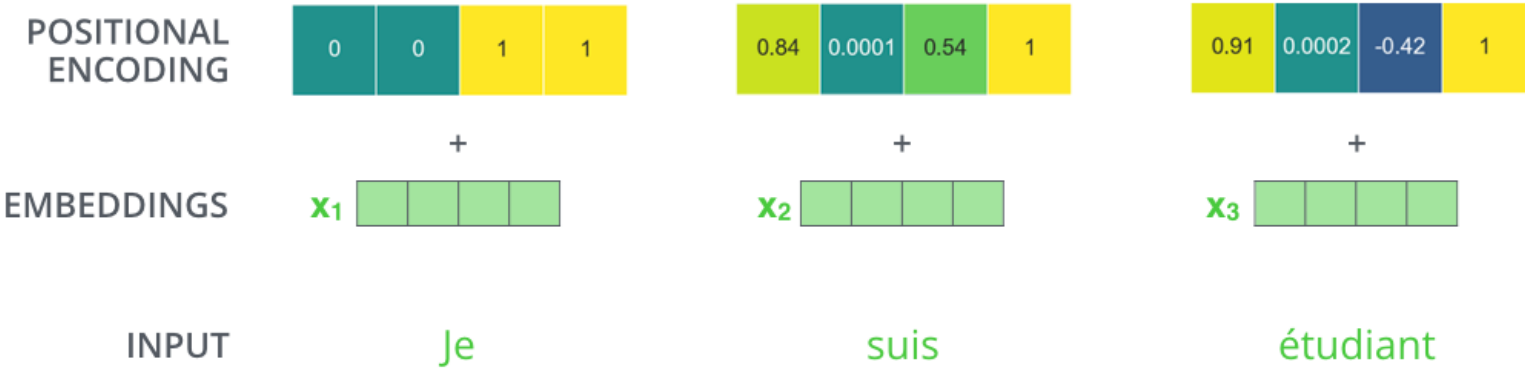
Challenges: Difficult to explain if we put all of them together!!! Position information is not really embedded.

Positional Encoding: Encode the order



Idea: Create an **additional vector** with the **same dimension** which helps it determine the position of each word or the distance between different words in the sequence.

An example of the positional encoding



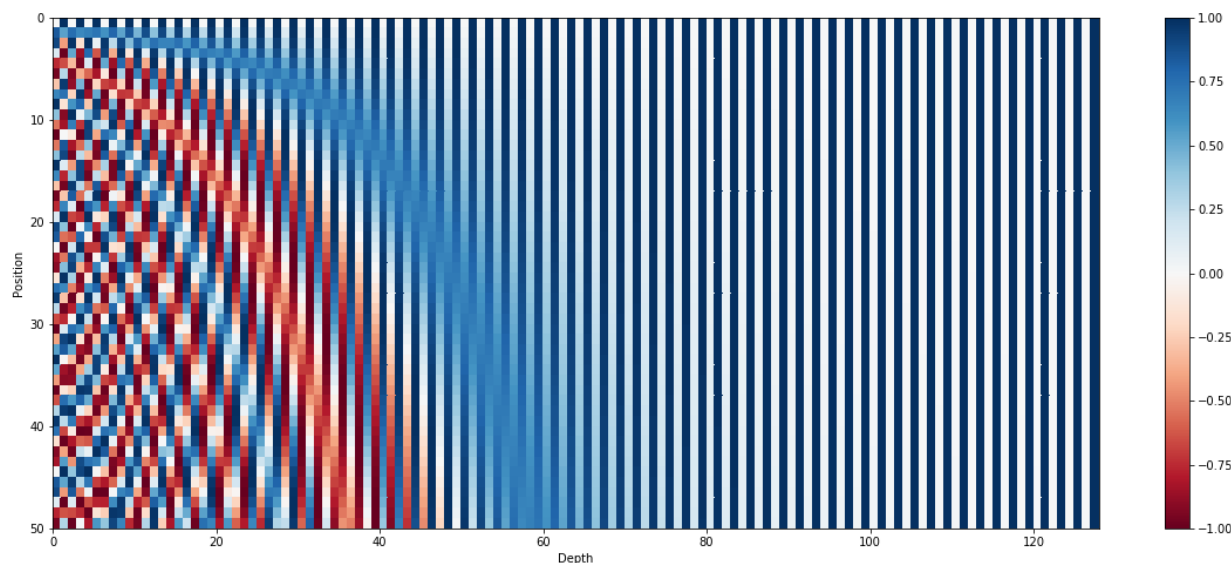
The positional encoding in the Attention paper

Let t be the desired position in an input sentence, $\vec{p}_t \in \mathbb{R}^d$ be its corresponding encoding, and d be the encoding dimension (where $d \equiv_2 0$)
Then $f : \mathbb{N} \rightarrow \mathbb{R}^d$ will be the function that produces the output vector $\vec{p}_t$ and it is defined as follows:

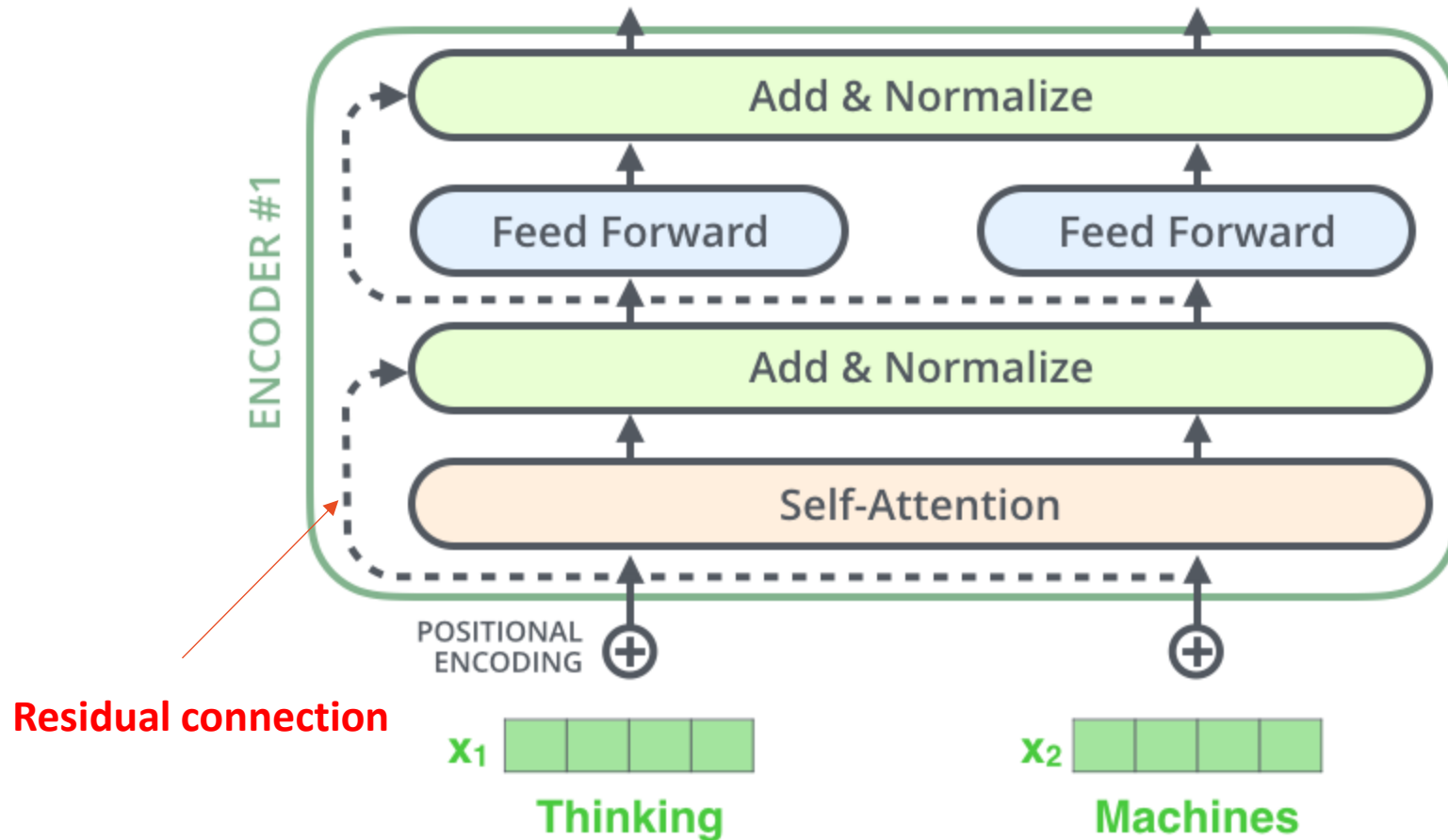
$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases}$$

where

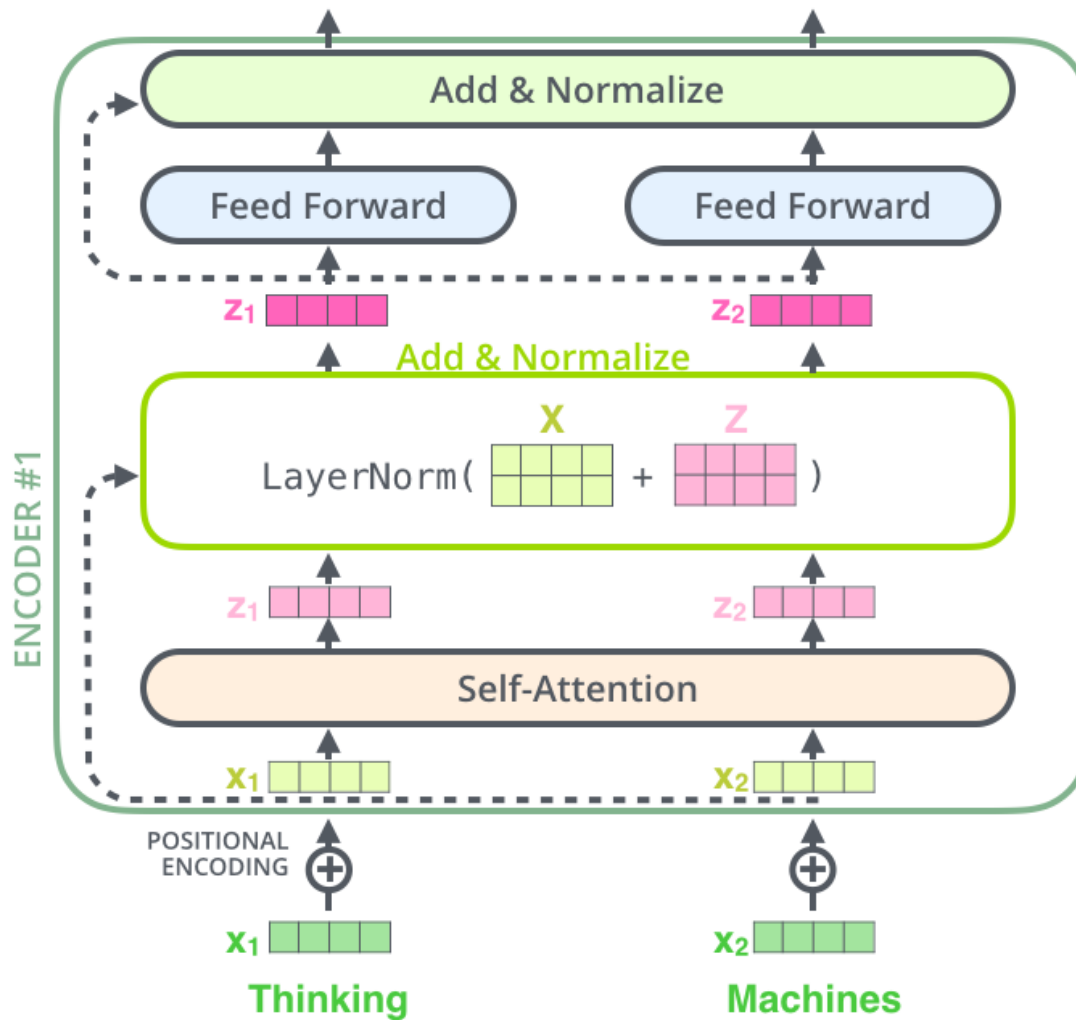
$$\omega_k = \frac{1}{10000^{2k/d}}$$



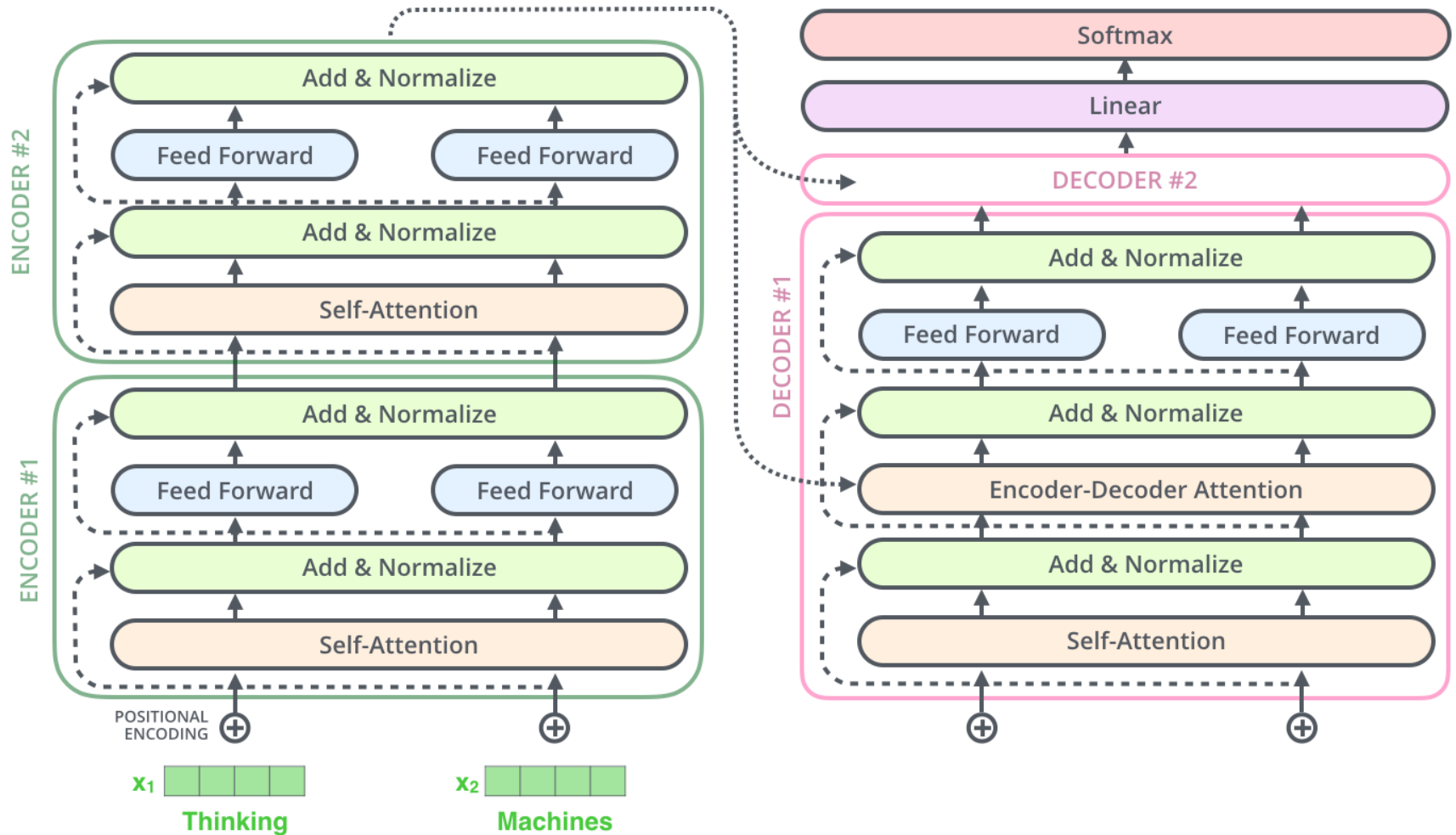
A residual connection in the encoders



An overview of the attention block



An overview of Transformer (two layers)

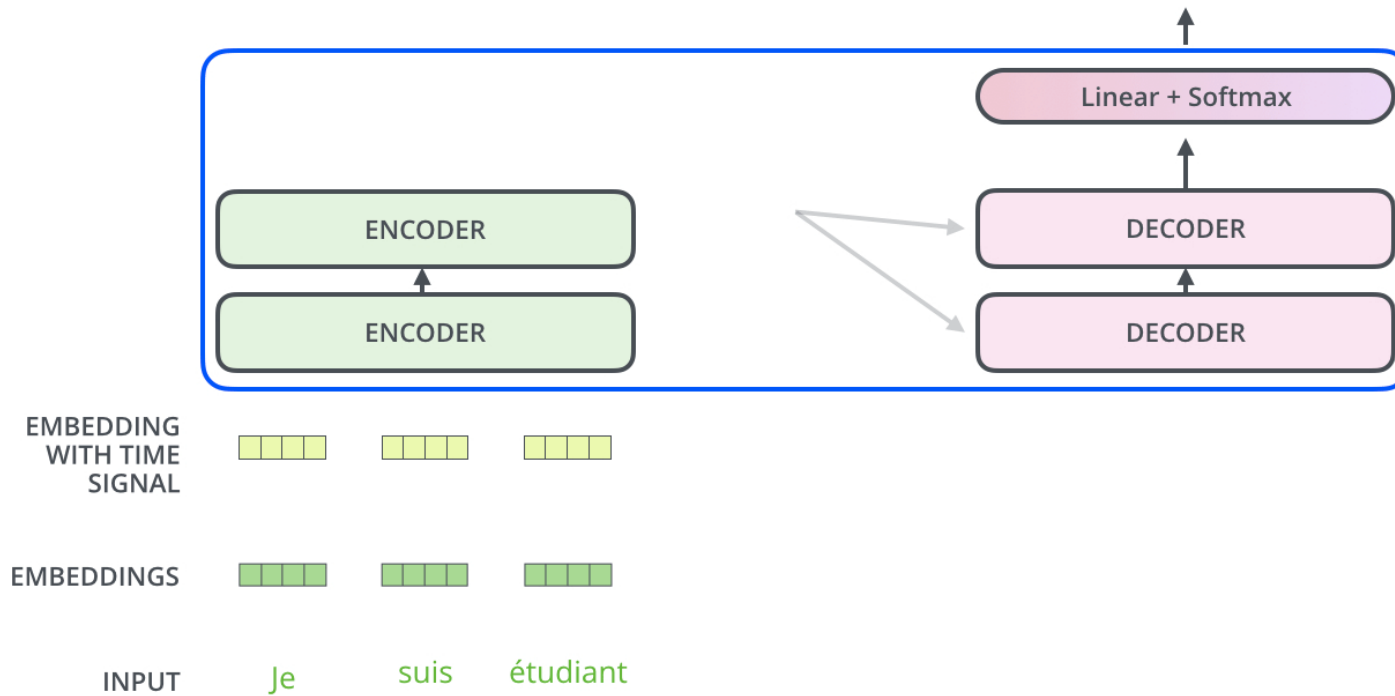


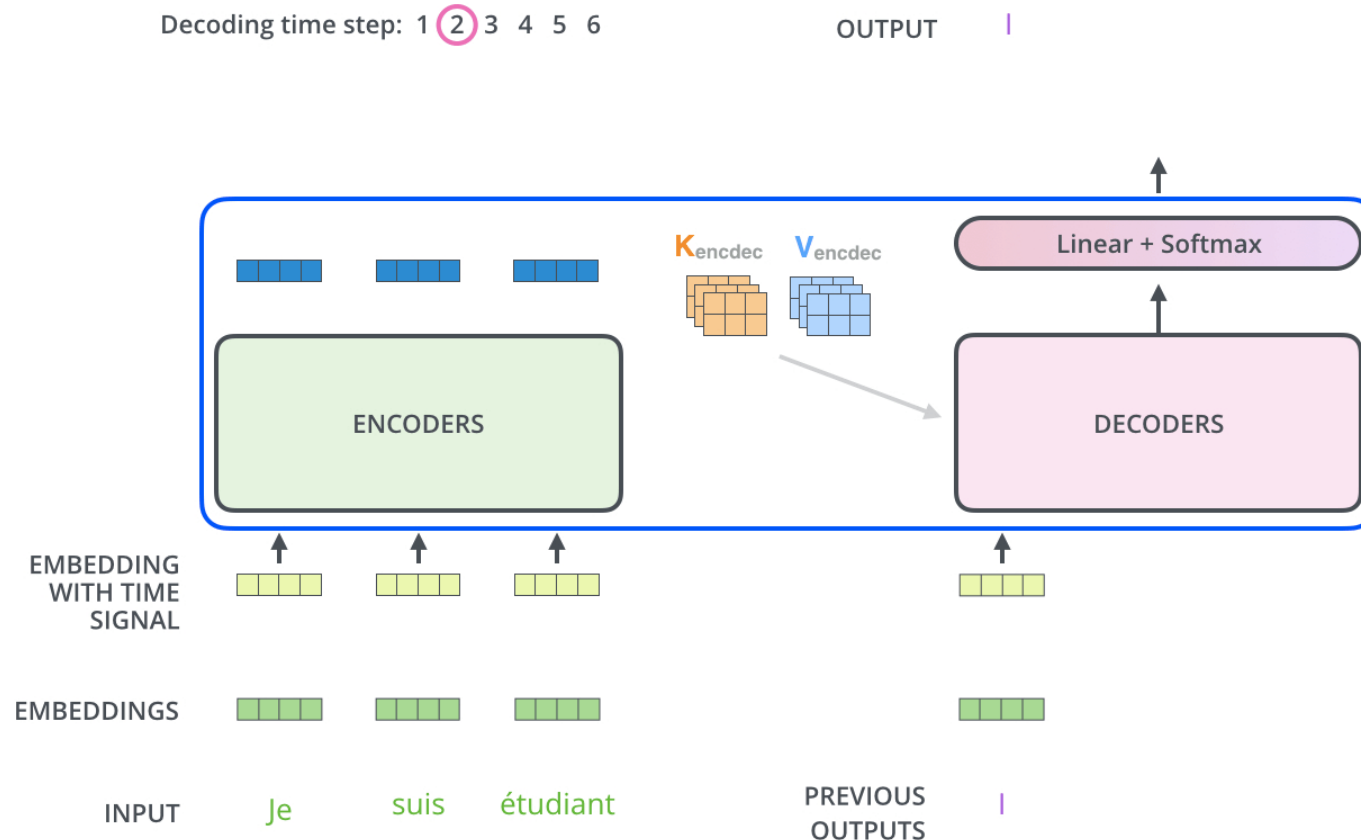
The decoder



Decoding time step: 1 2 3 4 5 6

OUTPUT





Warning: The self-attention layer is only allowed to attend to earlier positions in the output sequence. This is done by **masking future positions** (setting them to -inf) before the softmax step in the self-attention calculation.

Convert the output of the decoder to words



Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(**argmax**)

am

5

log_probs



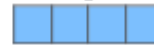
Softmax

logits



Linear

Decoder stack output





Train the Transformer

1. **English to German**: WMT 2014 English-German dataset (4.5 M sentence pairs).
Vocabulary token size: 37000.

2. **English to French**: WMT 2014 English-French dataset (36 M sentence pairs).
Vocabulary token size: 32000.

Training batch size: one batch contains about 25000 source tokens and 25000 target tokens.

Optimizer: **ADAM** with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$. The **learning rate** is scheduled as

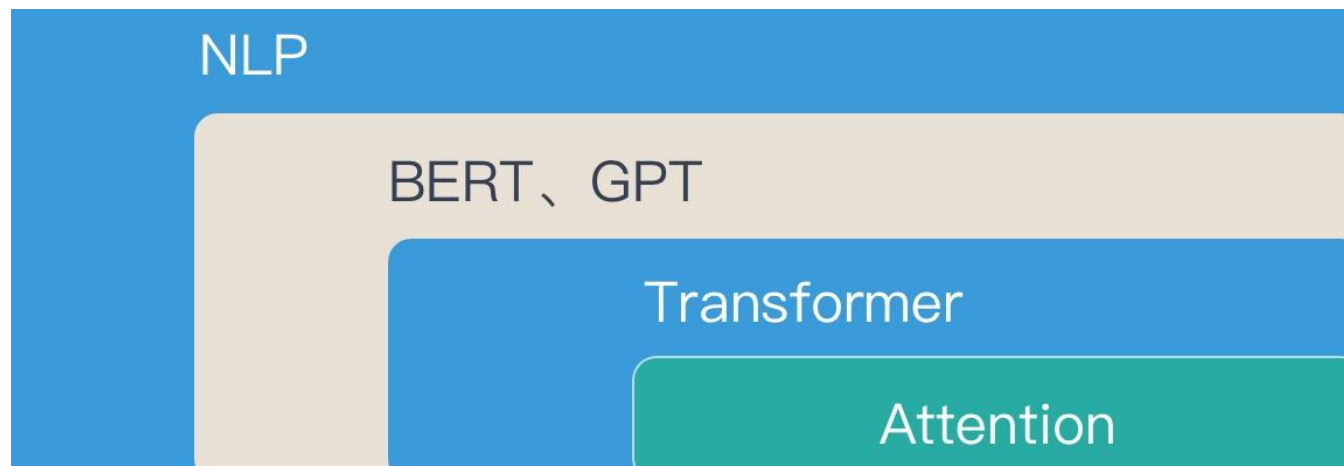
$$l_r = d_{model}^{-0.5} \times \min(num_{step}^{-0.5}, num_{step} \times num_{warmup}^{-1.5}),$$

where $num_{warmup} = 400$.

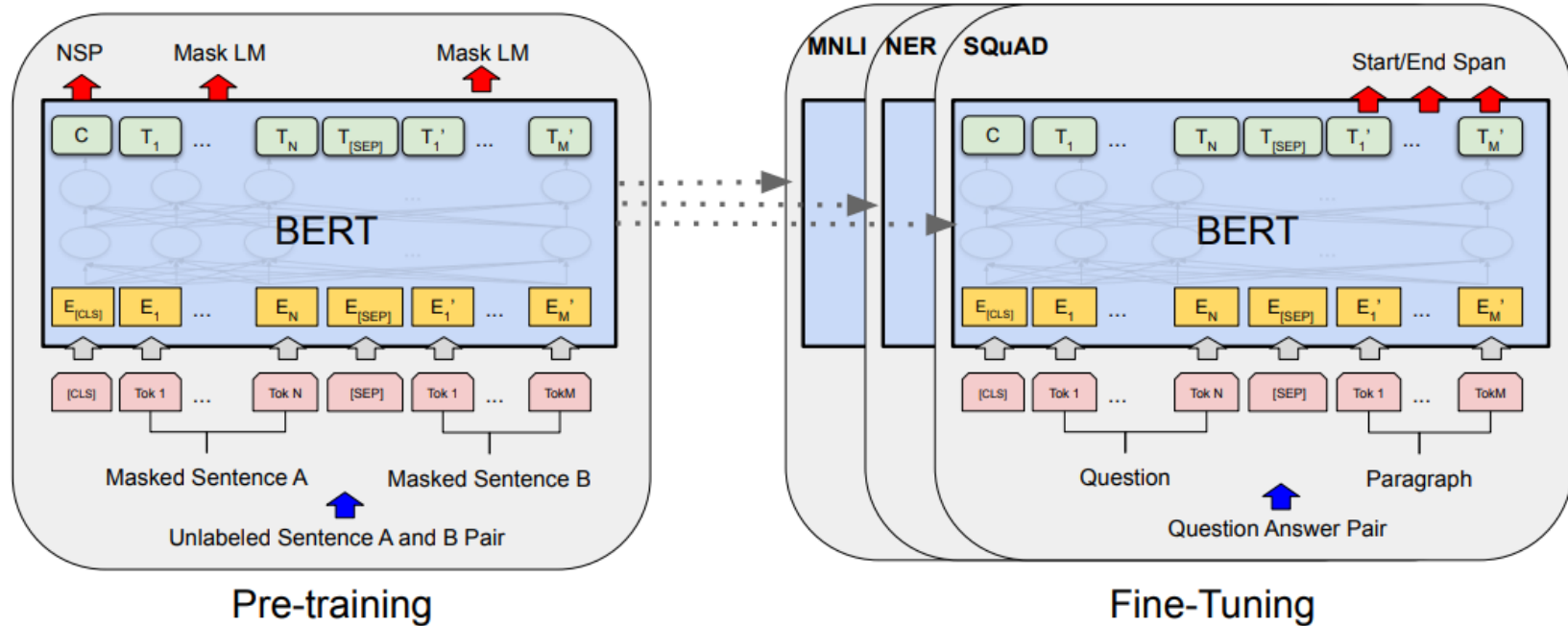
Regularization: **Dropout for each sublayer** with probability 0.1.

Applications

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. NAACL.



Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. NAACL.



Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33, 1877-1901.

